

SOLIDWORKS API (.NET VBA C#) — Examples (5/50): Analyze_Text_and_Geometry_in_G101_E1at_S- Change_Active_Tab_in_Manager_Pane_Exampl

SOLIDWORKS API Documentation (converted from CHM)

Analyze Text and Geometry in GTol Symbol Example (VB.NET)

This example shows how to analyze the text and geometry in a GTol symbol.

```

'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Gets whether <"GTOL-SPROF"> is bound.
' 3. Gets the number of lines, arcs, circles, text strings,
'    and triangles in <"GTOL-SPROF">.
' 4. Gets the number of point coordinates in the
'    lines in <"GTOL-SPROF">.
' 5. Gets the x,y,z coordinates for the first and second points
'    of each line in <"GTOL-SPROF">.
' 6. Gets the arc information for <"GTOL-SPROF">.
' 7. Examine the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swEnvironment As SolidWorks.Interop.sldworks.Environment
    Dim fileName As String
    Dim errors As Integer
    Dim warnings As Integer
    Dim nbrSymbols As Object
    Dim lineSymbols As Object
    Dim arcInfo As Object
    Dim nbrLineCoordinates As Integer
    Dim i As Integer

    Sub main()

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\tolanalyst\minimum_clearance\top_plate.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warn
        swEnvironment = swApp.GetEnvironment

        Debug.Print("GTOL library - Surface Profile symbol name:")

        'Get whether <"GTOL-SPROF"> is bound
        Dim bound As Boolean
        swEnvironment.GetIsBound("<GTOL-SPROF>", bound)
        Debug.Print(" Bound? " & bound)
        Debug.Print("")

        'Get the number of lines, arcs, circles, text strings,
        'and triangles in <"GTOL-SPROF">
        nbrSymbols = swEnvironment.GetSymEdgeCounts("<GTOL-SPROF>")
        Debug.Print(" Line count: " & nbrSymbols(0))
        Debug.Print(" Arc count: " & nbrSymbols(1))
        Debug.Print(" Circle count: " & nbrSymbols(2))
        Debug.Print(" Text count: " & nbrSymbols(3))
        Debug.Print(" Triangle count: " & nbrSymbols(4))
        Debug.Print("")
        Debug.Print("Number of lines: " & nbrSymbols(0))

        'Get the x,y,z coordinates for the first
        'and second points of each line
        lineSymbols = swEnvironment.GetSymLines("<GTOL-SPROF>")
        nbrLineCoordinates = UBound(lineSymbols)
        Debug.Print(" Number of point coordinates: " & nbrLineCoordinates + 1)
        Debug.Print(" Point coordinates:")
        Debug.Print(" (Line 1's x,y,z coordinates for first and second points,")
        Debug.Print(" Line 2's x,y,z coordinates for first and second points,")
        Debug.Print(" ...)")
        For i = 0 To nbrLineCoordinates
            Debug.Print(" " & lineSymbols(i))
        Next i
        Debug.Print("")
        Debug.Print("Number of arcs: " & nbrSymbols(1))

        'Get the x,y,z coordinates of the center, start, and
        'end points of each arc, whether the arc is filled,

```

```
'and whether a chord joining the start and end points
'of the arc is drawn
Debug.Print("  x,y,z coordinates of the center, start, and end points of each arc,")
Debug.Print("  whether the arc is filled (1 = true, 0 = false),")
Debug.Print("  and whether a chord is displayed for each arc (1 = true, 0 = false):")
arcInfo = swEnvironment.GetSymArcs2("<GTOL-SPROF>")
For i = 0 To UBound(arcInfo)
    Debug.Print("          " & arcInfo(i))
Next i
```

End Sub

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Anchor a Note Example (C#)

This example shows how to:

- insert a note.
- set a note's text to all uppercase and its vertical justification to bottom.
- anchor and position a note.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
//
// Postconditions:
// 1. Inserts a note.
// 2. Sets the note's text to all uppercase and its vertical justification
//    to bottom.
// 3. Locks the note at the specified position.
// 4. Examine the graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using System;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Note myNote;
        Annotation myAnnotation;
        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            myNote = (Note)Part.InsertNote("Drawing View1 Note");
            if ((myNote != null))
            {
                myNote.AllUpperCase = true;
                myNote.SetTextVerticalJustification((int)swTextAlignmentVertical_e.swTextAlignmentBottom);
                myNote.LockPosition = true;
                myAnnotation = (Annotation)myNote.GetAnnotation();
                if ((myAnnotation != null))
                {
                    boolstatus = myAnnotation.SetPosition(0.2, 0.269, 0);
                }
            }

            Part.ClearSelection2(true);
            Part.WindowRedraw();
        }
    }
    public SldWorks swApp;
}
```

Anchor a Note Example (VBA)

This example shows how to:

- insert a note.
- set a note's text to all uppercase and its vertical justification to bottom.
- anchor and position a note.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
'-----  
' Postconditions:  
' 1. Inserts a note.  
' 2. Sets the note's text to all uppercase and its vertical justification  
'    to bottom.  
' 3. Locks the note at the specified position.  
' 4. Examine the graphics area.  
'-----  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim myNote As SldWorks.Note  
Dim myAnnotation As SldWorks.Annotation  
Dim boolstatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
  
    Set myNote = Part.InsertNote("Drawing View1 Note")  
    If Not myNote Is Nothing Then  
        myNote.AllUpperCase = True  
        myNote.SetTextVerticalJustification (swTextAlignmentVertical_e.swTextAlignmentBottom)  
        myNote.LockPosition = True  
        Set myAnnotation = myNote.GetAnnotation()  
        If Not myAnnotation Is Nothing Then  
            boolstatus = myAnnotation.SetPosition(0.2, 0.269, 0)  
        End If  
    End If  
  
    Part.ClearSelection2 True  
    Part.WindowRedraw  
  
End Sub
```

Anchor a Note Example (VB.NET)

This example shows how to:

- insert a note.
- set a note's text to all uppercase and its vertical justification to bottom.
- anchor and position a note.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
'  
' Postconditions:  
' 1. Inserts a note.  
' 2. Sets the note's text to all uppercase and its vertical justification  
'    to bottom.  
' 3. Locks the note at the specified position.  
' 4. Examine the graphics area.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim Part As ModelDoc2  
    Dim myNote As Note  
    Dim myAnnotation As Annotation  
    Dim boolstatus As Boolean  
  
    Sub main()  
  
        Part = swApp.ActiveDoc  
        myNote = Part.InsertNote("Drawing View1 Note")  
        If Not myNote Is Nothing Then  
            myNote.AllUpperCase = True  
            myNote.SetTextVerticalJustification(swTextAlignmentVertical_e.swTextAlignmentBottom)  
            myNote.LockPosition = True  
            myAnnotation = myNote.GetAnnotation()  
            If Not myAnnotation Is Nothing Then  
                boolstatus = myAnnotation.SetPosition(0.2, 0.269, 0)  
            End If  
        End If  
        Part.ClearSelection2(True)  
        Part.WindowRedraw()  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```

Apply and Remove Texture By Body Display State (C#)

This example shows how to apply and remove texture to and from a body by a display state.

```
//-----
// Preconditions: Verify that the specified part to open
// and texture exist.
//
// Postconditions:
// 1. Opens the specified part and applies texture to the
// selected body.
// 2. In the IDE, click the Continue button
// at System.Diagnostics.Debugger.Break().
// 3. Removes texture from selected body.
// 4. To verify, click the Stop Debugging button in the
// IDE to stop execution of the macro and click anywhere
// in the graphics area.
// - or -
// If the Stop Debugging button is grayed out, click
// anywhere in the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace SetRemoveTextureByDisplayStateBodyCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Body2 body = default(Body2);
            Texture texture = default(Texture);
            ModelView modelview = default(ModelView);
            bool status = false;
            string displayState = null;
            int errors = 0;
            int warnings = 0;
            string namStr = null;

            // Open document and select a body
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\multibody\\multi_bridge.sldprt", (int)swDocumentTypes_e.swDocPART,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("hub", "SOLIDBODY", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            body = (Body2)swSelMgr.GetSelectedObject6(1, -1);

            // Set texture on selected body in the
            // specified display state
            displayState = "<Default>_Display State 1";
            namStr = "<SystemTexture>\\images\\textures\\pattern\\checker2.jpg";
            texture = (Texture)swModelDocExt.CreateTexture(namStr, 5, 45, false);
            status = body.SetTextureByDisplayState(displayState, texture);

            // Redraw the window view
            modelview = (ModelView)swModel.ActiveView;
            modelview.GraphicsRedraw(null);

            // Examine the selected body to verify
            // that the specified texture was set
            // In the IDE, click the Continue button to resume
            // running macro
            System.Diagnostics.Debugger.Break();
        }
    }
}
```

```

        // Remove texture from body by display state
        status = swModelDocExt.SelectByID2("hub", "SOLIDBODY", 0, 0, 0, false, 0, null, 0);
        body = (Body2)swSelMgr.GetSelectedObject6(1, -1);
        status = body.RemoveTextureByDisplayState(displayState);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;

}
}

```

Apply and Remove Texture By Body Display State Example (VBA)

This example shows how to add and remove texture to and from a body using the name of a display state of the model.

```
'-----
' Preconditions: Verify that the specified part to open and
' texture exist.
'
' Postconditions:
' 1. Opens part and applies texture to the selected body.
' 2. Examine the part and press F5.
' 3. Examine the part again to verify that the texture was removed.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim body As SldWorks.Body2
Dim texture As SldWorks.texture
Dim modelview As SldWorks.modelview
Dim status As Boolean
Dim displayState As String
Dim errors As Long
Dim warnings As Long
Dim namStr As String
Dim atIndex(1) As Long

Sub main()

    ' Open document and select a body
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_bridge.sldprt", swDocPART,
swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("hub", "SOLIDBODY", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set body = swSelMgr.GetSelectedObject6(1, -1)

    ' Set texture on selected body in the
    ' specified display state
    displayState = "<Default>_Display State 1"
    namStr = "<SystemTexture>\images\textures\pattern\checker2.jpg"
    Set texture = swModelDocExt.CreateTexture(namStr, 5, 45, False)
    status = body.SetTextureByDisplayState(displayState, texture)

    ' Redraw the window view
    Set modelview = swModel.ActiveView
    modelview.GraphicsRedraw (Empty)

    ' Examine the selected body to verify
    ' that the specified texture was set
    ' Press F5
    Stop

    ' Remove texture from body by display state
    status = swModelDocExt.SelectByID2("hub", "SOLIDBODY", 0, 0, 0, False, 0, Nothing, 0)
    Set body = swSelMgr.GetSelectedObject6(1, -1)
    status = body.RemoveTextureByDisplayState(displayState)

    ' Deselect the body to verify
    ' that the texture was removed
    atIndex(1) = 1
    status = swSelMgr.DeSelect2(atIndex, -1)

End Sub
```

Apply and Remove Texture By Body Display State Example (VB.NET)

This example shows how to add and remove texture to and from a body using the name of a display state of the model.

```
'-----
' Preconditions: Verify that the specified part to open
' and texture exist.
'
' Postconditions:
' 1. Opens the specified part and applies texture to the
'    selected body.
' 2. In the IDE, click the Continue button
'    at Stop.
' 3. Removes texture from selected body.
' 4. To verify, click the Stop Debugging button in the
'    IDE to stop execution of the macro and click anywhere
'    in the graphics area.
'    - or -
'    If the Stop Debugging button is grayed out, click
'    anywhere in the graphics area.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swModelDocExt As ModelDocExtension
        Dim body As Body2
        Dim texture As Texture
        Dim modelview As ModelView
        Dim status As Boolean
        Dim displayState As String
        Dim errors As Long
        Dim warnings As Long
        Dim namStr As String

        ' Open document and select a body
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\tutoria\multibody\multi_bridge.sldprt",
swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("hub", "SOLIDBODY", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        body = swSelMgr.GetSelectedObject6(1, -1)

        ' Set texture on selected body in the
        ' specified display state
        displayState = "<Default>_Display State 1"
        namStr = "<SystemTexture>\images\textures\pattern\checker2.jpg"
        texture = swModelDocExt.CreateTexture(namStr, 5, 45, False)
        status = body.SetTextureByDisplayState(displayState, texture)

        ' Redraw the window view
        modelview = swModel.ActiveView
        modelview.GraphicsRedraw(Nothing)

        ' Examine the selected body to verify
        ' that the specified texture was set
        ' In the IDE, click the Continue button to
        ' resume running macro
        Stop

        ' Remove texture from body by display state
        status = swModelDocExt.SelectByID2("hub", "SOLIDBODY", 0, 0, 0, False, 0, Nothing, 0)
```

```
body = swSelMgr.GetSelectedObject6(1, -1)  
status = body.RemoveTextureByDisplayState(displayState)
```

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Apply and Remove Texture To and From Component By Display State (C#)

This example shows how to apply and remove texture to and from a component using the name of a display state of the model.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly to open and texture exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly and applies texture to
//    the selected component.
// 2. Examine the Immediate window.
// 3. Click anywhere in the SOLIDWORKS Design window
//    to verify that the texture was applied.
// 4. Follow the instructions in the macro
//    to remove the just-applied texture.
// 5. Run the macro again.
// 6. Examine the Immediate window.
// 7. Click anywhere in the SOLIDWORKS Design window to verify
//    that the texture was removed.
//
// NOTE: Because this assembly document is used elsewhere,
// do not save changes.
//-----
using
```

SolidWorks.Interop.sldworks; using SolidWorks.Interop.swconst; using System; using System.Diagnostics; namespace

ApplyRemoveTextureByDisplayStateCSharp.csproj

```
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Component2 component = default(Component2);
            Texture texture = default(Texture);
            bool status = false;
            string displayState = null;
            int errors = 0;
            int warnings = 0;
            string namStr = null;

            // Open document and select a component
            swModel = (
                ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\motionstudies\\valve_cam.sldasm",
                (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (
                ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID(
                "rocker-1@valve_cam", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (
                SelectionMgr)swModel.SelectionManager;
            component = (
                Component2)swSelMgr.GetSelectedObject6(1, -1);

            displayState =
                "Default_Display State-1";
            namStr =
                "<SystemTexture>\\images\\textures\\pattern\\checker2.jpg";

            // Set texture on selected component in the
            // specified display state

            // After running the macro the first time, edit the macro to
            // insert the comment characters (//) before the following two
            // lines of code
            texture = (
                Texture)swModelDocExt.CreateTexture(namStr, 5, 45, false);
            Debug.Print ("Texture set: " + component.SetTextureByDisplayState(displayState, texture));
```

```
// Remove texture from component by display state

// After running the macro the first time, edit it to
// remove the comment characters (//) before the following
// line of code
//Debug.Print ("Texture removed: " + component.RemoveTextureByDisplayState(displayState));
    }

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}
```

Apply and Remove Texture By Component Display State Example (VBA)

This example shows how to add and remove texture to and from a component using the name of a display state of the assembly.

```
'-----
' Preconditions: Verify that the assembly to open and pattern exist.
'
' Postconditions:
' 1. Opens the specified assembly and applies texture to the
'    selected component.
' 2. At Stop, click the Rebuild button in SOLIDWORKS Design and examine the
'    component to verify that texture was applied.
' 3. In the IDE, click the Continue button to resume
'    execution of the macro.
' 4. Examine the component to verify that the texture
'    was removed.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim component As SldWorks.Component2
Dim texture As SldWorks.texture
Dim modelview As SldWorks.modelview
Dim status As Boolean
Dim displayState As String
Dim errors As Long
Dim warnings As Long
Dim namStr As String
Dim atIndex(1) As Long

Sub main()
    ' Open document and select a component
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\motionstudies\valve_cam.sldasm", swDocASSEMBLY, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("rocker-1@valve_cam", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set component = swSelMgr.GetSelectedObject6(1, -1)

    ' Set texture on selected component in the
    ' specified display state
    displayState = "Default_Display State-1"
    namStr = "<SystemTexture>\images\textures\pattern\checker2.jpg"
    Set texture = swModelDocExt.CreateTexture(namStr, 5, 45, False)
    status = component.SetTextureByDisplayState(displayState, texture)

    ' Click the Rebuild button in SOLIDWORKS Design to verify
    ' that the specified texture was set
    ' In the IDE, click the Continue button to resume
    ' running macro

    Stop

    ' Remove texture from component by display state
    status = swModelDocExt.SelectByID2("rocker-1@valve_cam", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set component = swSelMgr.GetSelectedObject6(1, -1)
    status = component.RemoveTextureByDisplayState(displayState)

    ' Deselect the component to verify
    ' that the texture was removed
    atIndex(1) = 1
    status = swSelMgr.DeSelect2(atIndex, -1)
End Sub
```

Apply and Remove Texture To and From Component By Display State Example (VB.NET)

This example shows how to apply and remove texture to and from a component using the name of a display state of the model.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly to open and texture exist.
' 2. Open the Immediate window.
```

```
' Postconditions:
' 1. Opens the specified assembly and applies texture to
'    the selected component.
' 2. Examine the Immediate window.
' 3. Click anywhere in the SOLIDWORKS Design window
'    to verify that the texture was applied.
' 4. Follow the instructions in the macro
'    to remove the just-applied texture.
' 5. Run the macro again.
' 6. Examine the Immediate window.
' 7. Click anywhere in the SOLIDWORKS Design window to verify
'    that the texture was removed.
```

```
' NOTE: Because this assembly document is used elsewhere,
' do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

Partial

Class SolidWorksMacro

```
Public Sub main()
    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swModelDocExt As ModelDocExtension
    Dim component As Component2
    Dim texture As Texture
    Dim status As Boolean
    Dim displayState As String
    Dim errors As Long
    Dim warnings As Long
    Dim namStr As String

    ' Open document and select a component
    swModel = swApp.OpenDoc6(
        "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\motionstudies\valve_cam.sldasm", swDocumentTypes_e.swDocASSEMBLY,
        swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID(
        "rocker-1@valve_cam", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swSelMgr = swModel.SelectionManager
    component = swSelMgr.GetSelectedObject6(1, -1)

    displayState = "Default_Display State-1"
    namStr = "<SystemTexture>\images\textures\pattern\checker2.jpg"

    ' Set texture on selected component in the
    ' specified display state

    ' After running the macro the first time, edit the macro to
    ' insert the comment character (') before the following two
    ' lines of code
    texture = swModelDocExt.CreateTexture(namStr, 5, 45,
        False)
    Debug.Print(
        "Texture set: " & component.SetTextureByDisplayState(displayState, texture))

    ' Remove texture from component by display state

    ' After running the macro the first time, edit it to
    ' remove the comment character (') before the following
```

' line of code

'Debug.Print("Texture removed: " & component.**RemoveTextureByDisplayState**(displayState))

End Sub

''' <summary>

''' The SldWorks swApp variable is pre-assigned for you.

''' </summary>

Public swApp As SldWorks

End

Class

Apply and Remove Texture Example (C#)

This example shows how to apply a texture to a face by display state and remove it by configuration.

```
//-----
// Preconditions: Verify that the specified part to open exists.
//
// Postconditions:
// 1. Opens the part and applies the texture to the selected face.
// 2. Press F5 to continue running the macro.
// 3. Examine the part to verify that the texture
//    was removed from the Default configuration.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace RemoveTexture2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        ModelDocExtension swModelDocExt;
        Face2 face;
        Texture texture;
        ModelView modelview;
        bool status;
        string displayState;
        int errors;
        int warnings;
        string namStr;

        int[] atIndex = new int[2];

        public void Main()
        {
            // Open the document and select a face
            swModel = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\pplate.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent,
"", ref errors, ref warnings);
            swModelDocExt = swModel.Extension;

            status = swModelDocExt.SelectByID("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, false, 0, null,
0);

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            face = (Face2)swSelMgr.GetSelectedObject6(1, -1);

            // Set the texture on the selected face in the specified display state
            displayState = "<Default> Display State 1";
            namStr = "<SystemTexture>\\images\\textures\\pattern\\checker2.jpg";
            texture = swModelDocExt.CreateTexture(namStr, 5, 45, false);
            status = face.SetTextureByDisplayState(displayState, texture);

            // Redraw the window view
            modelview = (ModelView)swModel.ActiveView;
            modelview.GraphicsRedraw(null);

            // Examine the selected face to verify that the specified texture was set
            // Press F5 to continue running macro

            System.Diagnostics.Debugger.Break();

            // Remove the texture from the face by the specified configuration
            status = swModelDocExt.SelectByID("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, false, 0, null,
0);

            face = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            status = face.RemoveTexture2("Default");

            // Deselect the face to verify that the texture was removed
            atIndex[1] = 1;
            errors = swSelMgr.DeSelect2(atIndex, -1);
        }

        public SldWorks swApp;
    }
}
```

Apply and Remove Texture Example (VBA)

This example shows how to apply a texture to a face by display state and remove it by configuration.

```

'-----
' Preconditions: Verify that the specified part to open and texture exist.
'
' Postconditions:
' 1. Opens the part and applies the texture to the selected face.
' 2. Press F5 to continue running the macro.
' 3. Examine the part to verify that the texture
'    was removed from the Default configuration.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim face As SldWorks.Face2
Dim texture As SldWorks.texture
Dim modelview As SldWorks.modelview
Dim status As Boolean
Dim displayState As String
Dim errors As Long
Dim warnings As Long
Dim namStr As String
Dim atIndex(1) As Long

Sub main()
    ' Open the document and select a face
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\pplate.sldprt",
swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    status = swModelDocExt.SelectByID2("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, False, 0, Nothing, 0)

    Set swSelMgr = swModel.SelectionManager
    Set face = swSelMgr.GetSelectedObject6(1, -1)

    ' Set the texture on the selected face in the specified display state
    displayState = "<Default>_Display State 1"
    namStr = "<SystemTexture>\images\textures\pattern\checker2.jpg"

    Set texture = swModelDocExt.CreateTexture(namStr, 5, 45, False)
    status = face.SetTextureByDisplayState(displayState, texture)

    ' Redraw the window view
    Set modelview = swModel.ActiveView
    modelview.GraphicsRedraw (Empty)

    ' Examine the selected face to verify that the specified texture was set
    ' Press F5 to continue running macro

    Stop

    ' Remove the texture from the face in the specified configuration
    status = swModelDocExt.SelectByID2("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, False, 0, Nothing, 0)
    Set face = swSelMgr.GetSelectedObject6(1, -1)
    status = face.RemoveTexture2("Default")

    ' Deselect the face to verify that the texture was removed
    atIndex(1) = 1
    status = swSelMgr.DeSelect2(atIndex, -1)
End Sub

```

Apply and Remove Texture Example (VB.NET)

This example shows how to apply a texture to a face by display state and remove it by configuration.

 ' Preconditions: Verify that the specified part to open and texture exist.
 '-----

' Postconditions:
 ' 1. Opens the part and applies the texture to the selected face.
 ' 2. Press F5 to continue running the macro.
 ' 3. Examine the part to verify that the texture
 ' was removed from the Default configuration.
 '-----

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

Partial Class SolidWorksMacro

```
    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swModelDocExt As ModelDocExtension
    Dim face As Face2
    Dim texture As Texture
    Dim modelview As ModelView
    Dim status As Boolean
    Dim displayState As String
    Dim errors As Integer
    Dim warnings As Integer
    Dim namStr As String
    Dim atIndex(1) As Integer

    Sub main()

        ' Open the document and select a face
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\pplate.sldprt",
        swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        status = swModelDocExt.SelectByID("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, False, 0, Nothing,
0)

        swSelMgr = swModel.SelectionManager
        face = swSelMgr.GetSelectedObject6(1, -1)

        ' Set the texture on the selected face in the specified display state
        displayState = "<Default>_Display State 1"
        namStr = "<SystemTexture>\images\textures\pattern\checker2.jpg"
        texture = swModelDocExt.CreateTexture(namStr, 5, 45, False)
        status = face.SetTextureByDisplayState(displayState, texture)

        ' Redraw the window view
        modelview = swModel.ActiveView
        modelview.GraphicsRedraw(Nothing)

        ' Examine the selected face to verify that the specified texture was set
        ' Press F5 to continue running macro

        Stop

        ' Remove the texture from the face in the specified configuration
        status = swModelDocExt.SelectByID("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, False, 0, Nothing,
0)

        face = swSelMgr.GetSelectedObject6(1, -1)
        status = face.RemoveTexture2("Default")

        ' Deselect the face to verify that the texture was removed
        atIndex(1) = 1
        errors = swSelMgr.DeSelect2(atIndex, -1)

    End Sub

    Public swApp As SldWorks

End Class
```

Apply and Remove Texture By Display State Example (C#)

This example shows how to apply and remove texture to and from a face by display state.

```
//-----
// Preconditions: Verify that the specified part and texture exist.
//
// Postconditions:
// 1. Opens the part and applies texture to the selected face.
// 2. Examine the part.
// 3. In the IDE, click the Continue button at
//    System.Diagnostics.Debugger.Break().
// 4. Removes the texture from the selected face.
// 5. To verify, click the Stop Debugging button in the IDE
//    to stop execution of the macro, then click anywhere
//    in the SOLIDWORKS Design graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace SetRemoveTextureByDisplayStateFaceCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Face2 face = default(Face2);
            Texture texture = default(Texture);
            ModelView modelview = default(ModelView);
            bool status = false;
            string displayState = null;
            int errors = 0;
            int warnings = 0;
            string namStr = null;

            // Open document and select a face
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\pplate.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent,
"", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension) swModel.Extension;
            status = swModelDocExt.SelectByID2("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, false, 0, null,
0);

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            face = (Face2) swSelMgr.GetSelectedObject6(1, -1);

            // Set texture on selected face in the
            // specified display state
            displayState = "<Default>_Display State 1";
            namStr = "<SystemTexture>\\images\\textures\\pattern\\checker2.jpg";
            texture = (Texture) swModelDocExt.CreateTexture(namStr, 5, 45, false);
            status = face.SetTextureByDisplayState(displayState, texture);

            // Redraw the window view
            modelview = (ModelView)swModel.ActiveView;
            modelview.GraphicsRedraw(null);

            // Examine the selected face to verify
            // that the specified texture was set
            // In the IDE, click the Continue button to resume
            // running macro
            System.Diagnostics.Debugger.Break();

            // Remove texture from face by display state
```

```
0);
    status = swModelDocExt.SelectByID2("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, false, 0, null,
    face = (Face2)swSelMgr.GetSelectedObject6(1, -1);
    status = face.RemoveTextureByDisplayState(displayState);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;

}
}
```

Apply and Remove Texture By Display State Example (VBA)

This example shows how to apply and remove texture to and from a face by display state.

 ' Preconditions: Verify that the specified part and texture exist.
 '

' Postconditions:
 ' 1. Opens the specified part and applies texture to the selected face.
 ' 2. Examine the part.
 ' 3. In the IDE, click the **Continue** button at Stop to resume execution
 ' of the macro.
 ' 4. Removes the texture from the face.
 ' 5. Examine the part to verify that the texture was removed.
 '-----

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim face As SldWorks.Face2
Dim texture As SldWorks.texture
Dim modelview As SldWorks.modelview
Dim status As Boolean
Dim displayState As String
Dim errors As Long
Dim warnings As Long
Dim namStr As String
Dim atIndex(1) As Long
```

Sub main()

' Open document and select a face

Set swApp = Application.SldWorks

Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\pplate.sldprt",
 swDocPART, swOpenDocOptions_Silent, "", errors, warnings)

Set swModelDocExt = swModel.Extension

status = swModelDocExt.SelectByID2("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, False, 0,
 Nothing, 0)

Set swSelMgr = swModel.SelectionManager

Set face = swSelMgr.GetSelectedObject6(1, -1)

' Set texture on selected face in the

' specified display state

displayState = "<Default>_Display State 1"

namStr = "<SystemTexture>\images\textures\pattern\checker2.jpg"

Set texture = swModelDocExt.CreateTexture(namStr, 5, 45, False)

status = face.SetTextureByDisplayState(displayState, texture)

' Redraw the window view

Set modelview = swModel.ActiveView

modelview.GraphicsRedraw (Empty)

' Examine the selected face to verify

' that the specified texture was set

' In the IDE, click the Continue button to resume

' running macro

Stop

' Remove texture from face by display state

status = swModelDocExt.SelectByID2("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, False, 0,
 Nothing, 0)

Set face = swSelMgr.GetSelectedObject6(1, -1)

status = face.RemoveTextureByDisplayState(displayState)

' Deselect the face to verify

```
' that the texture was removed  
atIndex(1) = 1  
status = swSelMgr.DeSelect2(atIndex, -1)
```

```
End Sub
```

Apply and Remove Texture By Display State Example (VB.NET)

This example shows how to apply and remove texture to and from a face by display state.

' Preconditions: Verify that the specified part and texture exist.

' Postconditions:

- ' 1. Opens the specified part and applies texture to the selected face.
 - ' 2. Examine the part.
 - ' 3. In the IDE, click the **Continue** button at Stop.
 - ' 4. Removes the texture from the selected face.
 - ' 5. To verify, click the **Stop Debugging** button in the IDE to stop execution of the macro, then click anywhere in the SOLIDWORKS Design graphics area.
-

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System

Partial Class SolidWorksMacro

Public Sub main()

Dim swModel As ModelDoc2

Dim swSelMgr As SelectionMgr

Dim swModelDocExt As ModelDocExtension

Dim face As Face2

Dim texture As Texture

Dim modelview As ModelView

Dim status As Boolean

Dim displayState As String

Dim errors As Long

Dim warnings As Long

Dim namStr As String

' Open document and select a face

swModel = swApp.**OpenDoc6**("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\pplate.sldprt",
swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

swModelDocExt = swModel.**Extension**

status = swModelDocExt.**SelectByID2**("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, False, 0, Nothing,
0)

swSelMgr = swModel.**SelectionManager**

face = swSelMgr.**GetSelectedObject6**(1, -1)

' Set texture on selected face in the

' specified display state

displayState = "<Default>_Display State 1"

namStr = "<SystemTexture>\images\textures\pattern\checker2.jpg"

texture = swModelDocExt.**CreateTexture**(namStr, 5, 45, False)

status = face.**SetTextureByDisplayState**(displayState, texture)

' Redraw the window view

modelview = swModel.**ActiveView**

modelview.**GraphicsRedraw**(Nothing)

' Examine the selected face to verify

' that the specified texture was set

' In the IDE, click the Continue button to

' resume running macro

Stop

' Remove texture from face by display state

status = swModelDocExt.**SelectByID2**("", "FACE", -0.02341747645642, 0.03900188771217, -0.008053400767039, False, 0, Nothing,
0)

face = swSelMgr.**GetSelectedObject6**(1, -1)

```
status = face.RemoveTextureByDisplayState(displayState)
```

```
End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Apply and Remove Texture To and From Model By Display State Example (C#)

This example shows how to apply and remove texture to and from a model by a display state.

```
//-----
// Preconditions:
// 1. Verify that the specified part and texture exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part and applies texture to the part.
// 2. Examine the part
// 3. In the IDE, click the Continue button at
//    System.Diagnostics.Debugger.Break().
// 4. Removes the texture from the selected face.
// 5. Examine the Immediate window and part.
//
// NOTE: Because the part is used elsewhere, do not save
// changes.
//-----

using

SolidWorks.Interop.sldworks;
using

SolidWorks.Interop.swconst;
using

System;
using

System.Diagnostics;
namespace

ApplyRemoveTextureByDisplayStateCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Texture texture = default(Texture);
            string displayState = null;
            int errors = 0;
            int warnings = 0;
            string namStr = null;

            // Open document
            swModel = (
                ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
                2018\\samples\\tutorial\\motionstudies\\valve.sldprt", (int)swDocumentTypes_e.swDocPART,
                (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (
                ModelDocExtension) swModel.Extension;

            // Set texture on model in the
            // specified display state
            displayState =
                "<Default>_Display State 1";
            namStr =
                "<SystemTexture>\\images\\textures\\pattern\\checker2.jpg";
            texture = (
                Texture) swModelDocExt.CreateTexture(namStr, 5, 45, false);
            Debug.Print("Texture set: " + swModelDocExt.SetTextureByDisplayState(displayState, texture));
            // Examine the selected face to verify
            // that the specified texture was set
            // In the IDE, click the Continue button to resume
            // running macro
            System.Diagnostics.Debugger.Break();
            // Remove texture from model by display state
            Debug.Print("Texture removed: " + swModelDocExt.RemoveTextureByDisplayState(displayState));

            // Rebuild model
            swModel.ForceRebuild3(
                false);
        }
    }
}
```

```
    }  
    /// <summary>  
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}  
}
```

Apply and Remove Texture To and From a Model by Display State Example (VBA)

This example shows how to add and remove texture to and from a model using the name of a display state model.

```
'-----
' Preconditions: Verify that the specified part and texture exist.
'
' Postconditions:
' 1. Opens the specified part and applies texture to the part.
' 2. At Stop, examine the part to verify that the texture was applied.
' 3. In the IDE, click the Continue button to resume
'    execution of macro.
' 4. Examine the part to verify that the texture was removed.
'
' NOTE: Because the part document is used elsewhere, do not save
' changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim texture As SldWorks.texture
Dim status As Boolean
Dim displayState As String
Dim errors As Long
Dim warnings As Long
Dim namStr As String

Sub main()

    ' Open document

Set swApp = Application.SldWorks
Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\motionstudies\valve.sldprt", swDocPART,
swOpenDocOptions_Silent, "", errors, warnings)

Set swModelDocExt = swModel.Extension

    ' Set
    texture on the model in the
    ' specified display state
    displayState = "<Default> Display State 1"
    namStr = "<SystemTexture>\images\textures\pattern\checker2.jpg"

Set texture = swModelDocExt.CreateTexture(namStr, 5, 45, False)

status = swModelDocExt.SetTextureByDisplayState(displayState, texture)

    ' Examine the model to verify texture applied

' In the IDE, click the Continue button to resume
'
running macro

    Stop

    ' Remove
    texture from model by display state
    status = swModelDocExt.RemoveTextureByDisplayState(displayState)

' Rebuild the model to verify texture was removed
swModel.ForceRebuild3 False

End Sub
```

Apply and Remove Texture To and From Model By Display State (VB.NET)

This example shows how to apply and remove texture to and from a model using the name of a display state of the model.

```
'-----
' Preconditions:
' 1. Verify that the specified part to open and texture exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part and applies texture to the part.
' 2. Examine the part to verify.
' 3. In the IDE, click the Continue button at Stop.
' 4. Removes the texture from the selected face.
' 5. Examine the Immediate window and the part.
'
' NOTE: Because the part document is used elsewhere, do not save changes.
'-----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System

Imports System.Diagnostics

Partial

Class SolidWorksMacro

Public Sub main()

Dim swModel As ModelDoc2

Dim swModelDocExt As ModelDocExtension

Dim texture As Texture

Dim displayState As String

Dim errors As Long

Dim warnings As Long

Dim namStr As String

' Open document

swModel = swApp.OpenDoc6(

"C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\motionstudies\valve.sldprt", swDocumentTypes_e.swDocPART,

swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

swModelDocExt = swModel.Extension

' Set texture on selected model in the

' specified display state

displayState =

"<Default>_Display State 1"

namStr =

"<SystemTexture>\images\textures\pattern\checker2.jpg"

texture = swModelDocExt.CreateTexture(namStr, 5, 45,

False)

Debug.Print(

"Texture set: " & swModelDocExt.SetTextureByDisplayState(displayState, texture))

' Examine the selected face to verify

' that the specified texture was set

' In the IDE, click the Continue button to

' resume running macro

Stop

' Remove texture from model by display state

Debug.Print("Texture removed: " & swModelDocExt.RemoveTextureByDisplayState(displayState))

swModel.ForceRebuild3(False)

End Sub

''' <summary>

''' The SldWorks swApp variable is pre-assigned for you.

''' </summary>

Public swApp As SldWorks

End
Class

Are the Assembly Configurations Loaded Example (C#)

This example shows how to find out if the configurations in an assembly are loaded, whether the configurations need to be updated and rebuilt, and the configuration types.

```
//-----
// Preconditions:
// 1. Verify that the assembly document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Loads all configurations.
// 2. Examine the Immediate window to see the states of the
//    configurations.
//
// NOTE: Because the specified assembly document is used elsewhere,
// do not save the document when you close it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace IsLoadedConfigurationCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Configuration swConfiguration = default(Configuration);
            ConfigurationManager swConfigurationMgr = default(ConfigurationManager);
            object[] ConfNameArr = null;
            string ConfName = null;
            const string DocFilename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\pdmworks\\speaker.sldasm";
            bool boolstatus = false;
            int Errors = 0;
            int Warnings = 0;
            int i;

            swModel = (ModelDoc2)swApp.OpenDoc6(DocFilename, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref Errors, ref Warnings);
            if (swModel == null)
            {
                return;
            }
            else
            {
                Debug.Print("File = " + swModel.GetPathName());
                Debug.Print("");
            }

            swConfigurationMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConfiguration = (Configuration)swConfigurationMgr.ActiveConfiguration;
            ConfNameArr = (object[])swModel.GetConfigurationNames();
            Debug.Print("Traverse assembly without activating other configurations...");
            for (i = 0; i < ConfNameArr.Length; i++)
            {
                ConfName = (string)ConfNameArr[i];
                swConfiguration = (Configuration)swModel.GetConfigurationByName(ConfName);
                Debug.Print(" Name of the configuration: " + swConfiguration.Name);
                Debug.Print(" Is the configuration loaded? " + swConfiguration.IsLoaded());
                Debug.Print(" Does the configuration need to be updated? " + swConfiguration.IsDirty());
                Debug.Print(" Does the configuration need to be rebuilt? " + swConfiguration.NeedsRebuild);
                Debug.Print(" What is the configuration type? " + swConfiguration.Type);
            }

            Debug.Print("");

            // Traverse the assembly again, but this time activate all
            // configurations, which loads them
            Debug.Print("Traverse assembly and activate all configurations...");
            for (i = 0; i < ConfNameArr.Length; i++)
            {
                ConfName = (string)ConfNameArr[i];
                swConfiguration = (Configuration)swModel.GetConfigurationByName(ConfName);
                boolstatus = swModel.ShowConfiguration2(ConfName);
                swConfiguration = (Configuration)swConfigurationMgr.ActiveConfiguration;
                Debug.Print(" Name of the configuration: " + swConfiguration.Name);
                Debug.Print(" Is the configuration loaded? " + swConfiguration.IsLoaded());
                Debug.Print(" Does the configuration need to be updated? " + swConfiguration.IsDirty());
                Debug.Print(" Does the configuration need to be rebuilt? " + swConfiguration.NeedsRebuild);
                Debug.Print(" What is the configuration type? " + swConfiguration.Type);
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Are the Assembly Configurations Loaded Example (VBA)

This example shows how to find out if the configurations in an assembly are loaded, whether the configurations need to be updated and rebuilt, and the configuration types.

```

'-----
' Preconditions:
' 1. Verify that the assembly document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Loads all configurations.
' 2. Examine the Immediate window to see the states of the
'    configurations.
'
' NOTE: Because the assembly is used elsewhere, do not save
' changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swConfiguration As SldWorks.Configuration
Dim swConfigurationMgr As SldWorks.ConfigurationManager
Dim vConfNameArr As Variant
Dim vConfName As Variant
Const sDocFilename As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\pdmworks\speaker.sldasm"
Dim boolstatus As Boolean
Dim nErrors As Long
Dim nWarnings As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Open document; exit if it doesn't open
    Set swModel = swApp.OpenDoc6(sDocFilename, swDocASSEMBLY, swOpenDocOptions_Silent, "", nErrors, nWarnings)
    If swModel Is Nothing Then
        Exit Sub
    Else
        Debug.Print "File = " & swModel.GetPathName
        Debug.Print ""
    End If

    Set swConfigurationMgr = swModel.ConfigurationManager
    Set swConfiguration = swConfigurationMgr.ActiveConfiguration
    vConfNameArr = swModel.GetConfigurationNames

    Debug.Print "Traverse assembly without activating other configurations..."
    For Each vConfName In vConfNameArr
        Set swConfiguration = swModel.GetConfigurationByName(vConfName)
        Debug.Print "    Name of the configuration: " & swConfiguration.Name
        Debug.Print "    Is the configuration loaded? " & swConfiguration.IsLoaded
        Debug.Print "    Does the configuration need to be updated? " & swConfiguration.IsDirty
        Debug.Print "    Does the configuration need to be rebuilt? " & swConfiguration.NeedsRebuild
        Debug.Print "    What is the configuration type? " & swConfiguration.Type
    Next

    Debug.Print ""

    ' Traverse the assembly again, but this time activate all
    ' configurations, which loads them
    Debug.Print "Traverse assembly and activate all configurations..."
    For Each vConfName In vConfNameArr
        Set swConfiguration = swModel.GetConfigurationByName(vConfName)
        boolstatus = swModel.ShowConfiguration2(vConfName)
        Set swConfiguration = swConfigurationMgr.ActiveConfiguration
        Debug.Print "    Name of the configuration: " & swConfiguration.Name
        Debug.Print "    Is the configuration loaded? " & swConfiguration.IsLoaded
        Debug.Print "    Does the configuration need to be updated? " & swConfiguration.IsDirty
        Debug.Print "    Does the configuration need to be rebuilt? " & swConfiguration.NeedsRebuild
        Debug.Print "    What is the configuration type? " & swConfiguration.Type
    Next

End Sub

```

Are the Assembly Configurations Loaded Example (VB.NET)

This example shows how to find out if the configurations in an assembly are loaded, whether the configurations need to be updated and rebuilt, and the configuration types.

```

-----
Preconditions:
1. Verify that the assembly document to open exists.
2. Open the Immediate window.
Postconditions:
1. Loads all configurations.
2. Examine the Immediate window to see the states of the
   configurations.
NOTE: Because the assembly is used elsewhere, do not save
changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swConfiguration As Configuration
        Dim swConfigurationMgr As ConfigurationManager
        Dim ConfNameArr As Object
        Dim ConfName As Object
        Const DocFilename As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\pdmworks\speaker.sldasm"
        Dim boolstatus As Boolean
        Dim Errors As Long
        Dim Warnings As Long

        ' Open document; exit if it doesn't open
        swModel = swApp.OpenDoc6(DocFilename, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", Errors, Warnings)
        If swModel Is Nothing Then
            Exit Sub
        Else
            Debug.Print("File = " & swModel.GetPathName)
            Debug.Print("")
            End If

            swConfigurationMgr = swModel.ConfigurationManager
            swConfiguration = swConfigurationMgr.ActiveConfiguration
            ConfNameArr = swModel.GetConfigurationNames

            Debug.Print("Traverse assembly without activating other configurations...")
            For Each ConfName In ConfNameArr
                swConfiguration = swModel.GetConfigurationByName(ConfName)
                Debug.Print("  Name of the configuration: " & swConfiguration.Name)
                Debug.Print("  Is the configuration loaded? " & swConfiguration.IsLoaded)
                Debug.Print("  Does the configuration need to be updated? " & swConfiguration.IsDirty)
                Debug.Print("  Does the configuration need to be rebuilt? " & swConfiguration.NeedsRebuild)
                Debug.Print("  What is the configuration type? " & swConfiguration.Type)
            Next

            Debug.Print("")

            ' Traverse the assembly again, but this time activate all
            ' configurations, which loads them
            Debug.Print("Traverse assembly and activate all configurations...")
            For Each ConfName In ConfNameArr
                swConfiguration = swModel.GetConfigurationByName(ConfName)
                boolstatus = swModel.ShowConfiguration2(ConfName)
                swConfiguration = swConfigurationMgr.ActiveConfiguration
                Debug.Print("  Name of the configuration: " & swConfiguration.Name)
                Debug.Print("  Is the configuration loaded? " & swConfiguration.IsLoaded)
                Debug.Print("  Does the configuration need to be updated? " & swConfiguration.IsDirty)
                Debug.Print("  Does the configuration need to be rebuilt? " & swConfiguration.NeedsRebuild)
                Debug.Print("  What is the configuration type? " & swConfiguration.Type)
            Next

        End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Assign Tracking ID Using Macro Feature Example (VBA)

This example shows how to assign a tracking ID to a face using a macro feature, which then allows the tracking ID to be persistent across SOLIDWORKS Design sessions.

```

'-----
' Preconditions:
' 1. Click Tools > References > SolidWorks version exposed type libraries for  

'   add-in use > OK.  

' 2. Copy Macros to the main module and rename it Macros. (To rename the module, click  

'   View > Properties Window and type Macros in (Name).)  

' 3. Click Insert > Module and copy PropMgrConst to that module and name it PropMgrConst.  

' 4. Click Insert > Class module and copy PropMgr to that class module and name it PropMgr.  

' 5. Click Insert > Class module and copy PropMgrCmd to that class module and name it PropMgrCmd.  

' 6. Click Insert > Class module and copy PropMgrHdlr to that class module and name it PropMgrHdlr.  

' 7. Open a part.  

'-----
' Postconditions:
' 1. Select a face.  

' 2. Assigns a tracking ID of 2 to the selected face via a macro feature.  

' 3. Creates the macro feature CustomTrackingID1.  

' 4. Examine the FeatureManager design tree.
'-----

```

'Macros

Option Explicit

```

' Handle to Macro feature regeneration
Function swmMain(swAppIn, partIn, featureIn)
    Dim featData As MacroFeatureData
    Set featData = featureIn.GetDefinition

    Dim sels, types, selmarks
    Dim faceSel As Object
    Call featData.GetSelections(sels, types, selmarks)
    If IsEmpty(sels) Then
        swmMain = "Face was not been selected."
        Exit Function
    End If

    Set faceSel = sels(0)
    Dim TrackingID As Long
    Call featData.GetIntegerByName("TrackingID", TrackingID)

    If Not faceSel Is Nothing Then
        Dim Cookies As Long
        Cookies = swAppIn.RegisterTrackingDefinition("API_TrackingIDUsingMacroFeature")
        faceSel.setTrackingID Cookies, TrackingID
    End If
End Function

' Handle to Macro feature edit definition
Sub swmPM(swAppIn, partIn, featureIn)
    Dim swPage As New PropMgr
    swPage.Init swAppIn, partIn, featureIn, swCmdEdit, swAppIn.GetCurrentMacroPathName
    swPage.Show
End Sub

'Run this procedure to insert macro feature with customized Property Manager Page
Public Sub swmInsertCustomizedMacroFeature()
    Dim swAppIn, partIn, featureIn
    Set swAppIn = Application.SldWorks
    Set partIn = swAppIn.ActiveDoc

    Dim swPage As New PropMgr
    swPage.Init swAppIn, partIn, featureIn, swCmdCreate, swAppIn.GetCurrentMacroPathName
    swPage.Show
End Sub

```

[Back to top](#)

'PropMgrConst

```

Public Enum swPageCmdState
    swCmdCreate = 1
    swCmdEdit = 2
End Enum

Public Const ID_GROUP = 1
Public Const ID_SELECTION = 2
Public Const ID_NUMBER = 3

```

[Back to top](#)

'PropMgr

Option Explicit

```

Private m_swApp As SldWorks.SldWorks
Private m_Part As SldWorks.ModelDoc2
Private m_feature As SldWorks.feature
Private m_Page As PropertyManagerPage2
Private m_Group As PropertyManagerPageGroup
Private m_Selection As PropertyManagerPageSelectionbox
Private m_Number As PropertyManagerPageNumberbox

```

```

Private m_swPageCmd As New PropMgrCmd
Private m_cmdState As swPageCmdState
Private m_pageHdlr As New PropMgrHdlr

Private Sub Layout()
    Dim swPage As PropertyManagerPage2
    Dim swControl As PropertyManagerPageControl
    Dim title As String
    Dim buttonTypes As Long
    Dim message As String
    Dim ID As Long
    Dim controlType As Integer
    Dim caption As String
    Dim alignment As Integer
    Dim options As Long
    Dim tip As String
    Dim filterArray(0 To 0) As Long

    m_pageHdlr.Init Me

    If m_cmdState = swCmdCreate Then
        title = "Customized Macro"
    Else
        title = m_feature.Name
    End If
    buttonTypes = swPropertyManagerOptions_OkayButton + swPropertyManagerOptions_CancelButton + swPropertyManagerOptions_LockedPage
    Dim errorh As Long
    Set m_Page = m_swApp.CreatePropertyManagerPage(title, buttonTypes, m_pageHdlr, errorh)
    If Not m_Page Is Nothing Then

        'Initial set up of the dialog.
        message = "Customized Macro Feature."
        m_Page.SetMessage message, swImportantMessageBox

        'Begin adding the required controls to the dialog.

        'GROUP BOX -----
        ID = ID_GROUP
        caption = "Face Tracking ID"
        options = swGroupBoxOptions_Visible + swGroupBoxOptions_Expanded + swGroupBoxOptions_Checked
        Set m_Group = m_Page.AddGroupBox(ID, caption, options)
        If Not m_Group Is Nothing Then

            'CONTROL Selection box -----
            ID = ID_SELECTION
            controlType = swControlType_Selectionbox
            caption = "Sample selection box"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Select face"
            Set swControl = m_Group.AddControl(ID, controlType, caption, alignment, options, tip)
            If Not swControl Is Nothing Then
                Set m_Selection = swControl
                filterArray(0) = swSelFACES
                m_Selection.SingleEntityOnly = True
                m_Selection.Height = 50
                m_Selection.SetSelectionFilters (filterArray)
                m_Selection.SetStandardPictureLabel swBitmapLabel_SelectFaceSurface
            End If

            'CONTROL Number box -----
            ID = ID_NUMBER
            controlType = swControlType_Numberbox
            caption = "Sample number box"
            alignment = swControlAlign_Indent
            options = swControlOptions_Visible + swControlOptions_Enabled
            tip = "Face Tracking ID"
            Set swControl = m_Group.AddControl(ID, controlType, caption, alignment, options, tip)
            If Not swControl Is Nothing Then
                Set m_Number = swControl
                m_Number.SetRange swNumberBox_UnitlessInteger, 1#, 1000000#, 1, True

                m_Number.Value = m_swPageCmd.GetUserTrackingID()
                m_Number.SetStandardPictureLabel
            End If
        End If
    End If
End Sub

Public Sub Show()
    m_Page.Show
End Sub

Sub Init(swApp, part, feature, cmdState As swPageCmdState, macroPath As String)
    Set m_swApp = swApp
    Set m_Part = part
    If Not IsEmpty(feature) Then
        Set m_feature = feature
    End If
    m_cmdState = cmdState
    m_Part.ClearSelection
    m_swPageCmd.Init swApp, part, feature, cmdState, macroPath
    Layout
End Sub

Public Function GetCmd() As PropMgrCmd
    Set GetCmd = m_swPageCmd
End Function

```

[Back to top](#)

PropMgrCmd

Option Explicit

Private m_swApp As SldWorks.SldWorks

```

Private m_Part As SldWorks.ModelDoc2
Private m_feature As SldWorks.feature
Private m_faceSel As Object
Private m_TrackingID As Long
Private m_featData As SldWorks.MacroFeatureData
Private m_modelComp As SldWorks.Component2
Private m_cmdState As swPageCmdState
Private m_macroPath As String

Public Sub Init(swApp, part, feature, cmdState As swPageCmdState, macroPath As String)
    Set m_swApp = swApp
    Set m_Part = part
    m_macroPath = macroPath
    If Not IsEmpty(feature) Then
        Set m_feature = feature
    End If
    If cmdState = swCmdEdit Then ' On Edit Definition
        Dim ret As Boolean
        Set m_featData = m_feature.GetDefinition
        Set m_modelComp = m_feature.GetComponent
        m_cmdState = cmdState
        ret = m_featData.AccessSelections(m_Part, m_modelComp)
        Dim sels, types, selmarks
        Call m_featData.GetSelections(sels, types, selmarks)
        If Not IsEmpty(sels) Then
            Set m_faceSel = sels(0)
            Call m_faceSel.Select2(True, selmarks(0))
        End If
        Call m_featData.GetIntegerByName("TrackingID", m_TrackingID)
    Else ' On Insert Feature
        m_TrackingID = 2
    End If
End Sub

Public Sub OnOk()
    If m_cmdState = swCmdEdit Then ' On Edit Definition
        Dim sels, types, selmarks
        Call m_featData.GetSelections(sels, types, selmarks)
        Dim newSels(0 To 0) As Object
        Dim newSelMarks(0 To 0) As Long
        Set newSels(0) = m_faceSel
        newSelMarks(0) = 0
        sels = newSels
        selmarks = newSelMarks
        Call m_featData.SetSelections(sels, selmarks)
        Call m_featData.SetIntegerByName("TrackingID", m_TrackingID)
        Call m_feature.ModifyDefinition(m_featData, m_Part, m_modelComp)
    Else ' On Insert feature
        Dim paramNames, paramTypes, paramValues
        Dim paramNameArray(0 To 0) As String
        Dim paramTypeArray(0 To 0) As Long
        Dim paramValueArray(0 To 0) As String
        paramNameArray(0) = "TrackingID"
        paramTypeArray(0) = swMacroFeatureParamTypeInteger
        paramValueArray(0) = Str(m_TrackingID)
        paramNames = paramNameArray
        paramTypes = paramTypeArray
        paramValues = paramValueArray
        Dim methods(0 To 8) As String
        methods(0) = m_macroPath
        methods(1) = "Macros"
        methods(2) = "swmMain"
        methods(3) = m_macroPath
        methods(4) = "Macros"
        methods(5) = "swmPM"
        methods(6) = ""
        methods(7) = ""
        methods(8) = ""
        Dim feat As Object
        Set feat = m_Part.FeatureManager.InsertMacroFeature("CustomTrackingID", "", methods, (paramNames), (paramTypes), (paramValues), Nothing, swMacroFeatureByDefault)
    End If
End Sub

Public Sub OnCancel()
    If m_cmdState = swCmdEdit Then
        m_featData.ReleaseSelectionAccess
    End If
End Sub

Public Sub OnFaceSelect()
    Dim selM
    Set selM = m_Part.SelectionManager
    Set m_faceSel = Nothing
    Set m_faceSel = selM.GetSelectedObject3(1)
End Sub

Public Function getUserTrackingID() As Integer
    getUserTrackingID = m_TrackingID
End Function

Public Sub setUserTrackingID(TrackingID As Integer)
    m_TrackingID = TrackingID
End Sub

```

[Back to top](#)

'PropMgrHdlr

Option Explicit
Implements PropertyManagerPage2Handler

Dim m_pageObj As PropMgr

```

Public Sub Init(pageObj As PropMgr)
    Set m_pageObj = pageObj
End Sub

Private Function PropertyManagerPage2Handler_ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Long) As Boolean
End Function

Private Sub PropertyManagerPage2Handler_OnButtonPress(ByVal ID As Long)
End Sub

Private Sub PropertyManagerPage2Handler_OnClose(ByVal reason As Long)
    If reason = swPropertyManagerPageClose_Okay Then
        m_pageObj.GetCmd().OnOk
    ElseIf reason = swPropertyManagerPageClose_Cancel Then
        m_pageObj.GetCmd().OnCancel
    End If
End Sub

Private Sub PropertyManagerPage2Handler_OnCheckboxCheck(ByVal ID As Long, ByVal Checked As Boolean)
End Sub

Private Sub PropertyManagerPage2Handler_OnComboboxSelectionChanged(ByVal ID As Long, ByVal Item As Long)
End Sub

Private Sub PropertyManagerPage2Handler_OnGroupCheck(ByVal ID As Long, ByVal Checked As Boolean)
End Sub

Private Sub PropertyManagerPage2Handler_OnGroupExpand(ByVal ID As Long, ByVal Expanded As Boolean)
End Sub

Private Function PropertyManagerPage2Handler_OnHelp() As Boolean
End Function

Private Sub PropertyManagerPage2Handler_OnListboxSelectionChanged(ByVal ID As Long, ByVal Item As Long)
End Sub

Private Sub PropertyManagerPage2Handler_AfterClose()
    Set m_pageObj = Nothing
End Sub

Private Sub PropertyManagerPage2Handler_OnNumberboxChanged(ByVal ID As Long, ByVal Value As Double)
    m_pageObj.GetCmd().setUserTrackingID (Value)
End Sub

Private Sub PropertyManagerPage2Handler_OnOptionCheck(ByVal ID As Long)
End Sub

Private Sub PropertyManagerPage2Handler_OnSelectionboxFocusChanged(ByVal ID As Long)
End Sub

Private Sub PropertyManagerPage2Handler_OnTextboxChanged(ByVal ID As Long, ByVal Text As String)
End Sub

Private Sub PropertyManagerPage2Handler_OnSelectionBoxListChanged(ByVal ID As Long, ByVal Text As Long)
    m_pageObj.GetCmd().OnFaceSelect
End Sub

```

[Back to top](#)

Attach Annotation Example (C#)

This example shows how to attach an existing annotation to a drawing view.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\replaceview.slddrw.
//
// Postconditions:
// 1. Inserts a note annotation n the drawing.
// 2. Selects the annotation.
// 3. Appends a face in a drawing view to the selection list.
// 4. Attaches the annotation to the selected face.
// 5. Examine the drawing.
// 6. Close the drawing without saving it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace AttachAnnotation_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 part;
        DrawingDoc draw;
        Note aNote;
        Annotation anAnnot;
        SelectData selectData = null;
        int ret;

        bool boolstatus;

        public void Main()
        {
            part = (ModelDoc2)swApp.ActiveDoc;
            draw = (DrawingDoc)part;

            boolstatus = draw.ActivateSheet("Sheet1");

            aNote = (Note)draw.CreateText2("This is a note.", 0.21, 0.12, 0, 0.005, 0);
            anAnnot = (Annotation)aNote.GetAnnotation();
            ret = anAnnot.SetLeader3(swLeaderStyle_e.swBENT, swLeaderSide_e.swLS_SMART, true, false, false, false);

            anAnnot.Select3(false, selectData);
            boolstatus = draw.ActivateView("Drawing View1");
            boolstatus = part.Extension.SelectByID("", "FACE", 0.0783563575357558, 0.17448024010205, -499.965138294658, true, 0,
null, 0);

            draw.AttachAnnotation(swAttachAnnotationOption_e.swAttachAnnotationOption_View);
        }

        public SldWorks swApp;
    }
}
```

Attach Annotation Example (VBA)

This example shows how to attach an existing annotation to a drawing view.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\replaceview.slddrw.
'
' Postconditions:
' 1. Inserts a note annotation in the drawing.
' 2. Selects the annotation.
' 3. Appends a face in a drawing view to the selection list.
' 4. Attaches the annotation to the selected face.
' 5. Examine the drawing.
' 6. Close the drawing without saving it.
'-----

Dim swApp As SldWorks.SldWorks
Dim part As SldWorks.ModelDoc2
Dim draw As SldWorks.DrawingDoc
Dim aNote As SldWorks.Note
Dim anAnnot As SldWorks.Annotation
Dim selectData As SldWorks.selectData
Dim ret As Long
Dim boolstatus As Boolean
Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set part = swApp.ActiveDoc
    Set draw = part

    boolstatus = part.ActivateSheet("Sheet1")

    Set aNote = draw.CreateText2("This is a note.", 0.21, 0.12, 0, 0.005, 0)
    Set anAnnot = aNote.GetAnnotation
    ret = anAnnot.SetLeader3(swLeaderStyle_e.swBENT, swLeaderSide_e.swLS_SMART, True, False, False, False)

    anAnnot.Select3 False, selectData
    boolstatus = part.ActivateView("Drawing View1")
    boolstatus = part.Extension.SelectByID2("", "FACE", 7.83563575357558E-02, 0.17448024010205, -499.965138294658, True, 0,
    Nothing, 0)

    draw.AttachAnnotation swAttachAnnotationOption_e.swAttachAnnotationOption_View
End Sub

```

Attach Annotation Example (VB.NET)

This example shows how to attach an existing annotation to a drawing view.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\replaceview.slddrw.
'
' Postconditions:
' 1. Inserts a note annotation in the drawing.
' 2. Selects the annotation.
' 3. Appends a face in a drawing view to the selection list.
' 4. Attaches the annotation to the selected face.
' 5. Examine the drawing.
' 6. Close the drawing without saving it.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```

Partial Class SolidWorksMacro

```

```

    Dim part As ModelDoc2
    Dim draw As DrawingDoc
    Dim aNote As Note
    Dim anAnnot As Annotation
    Dim selectData As SelectData
    Dim ret As Integer
    Dim boolstatus As Boolean

```

```

    Sub main()

```

```

        part = swApp.ActiveDoc
        draw = part

```

```

        boolstatus = part.ActivateSheet("Sheet1")

```

```

        aNote = draw.CreateText2("This is a note.", 0.21, 0.12, 0, 0.005, 0)

```

```

        anAnnot = aNote.GetAnnotation

```

```

        ret = anAnnot.SetLeader3(swLeaderStyle_e.swBENT, swLeaderSide_e.swLS_SMART, True, False, False, False)

```

```

        anAnnot.Select3(False, selectData)

```

```

        boolstatus = part.ActivateView("Drawing View1")

```

```

        boolstatus = part.Extension.SelectByID2("", "FACE", 0.0783563575357558, 0.17448024010205, -499.965138294658, True, 0,
Nothing, 0)

```

```

        draw.AttachAnnotation(swAttachAnnotationOption_e.swAttachAnnotationOption_View)

```

```

    End Sub

```

```

    Public swApp As SldWorks

```

```

End Class

```

Attach Annotation to Entity Example (C#)

This example shows how to attach a note to a different entity.

```
//-----
// Preconditions:
// 1. Open a part or drawing that has at least one
//    annotation that is a note.
// 2. Select the note.
// 3. Press Ctrl while selecting a face, edge, or vertex.
//
// Postconditions:
// 1. Attaches the selected note to the entity
//    selected in Preconditions step 3, if possible.
// 2. Examine the note in the graphics area.
//
// NOTE: If you open a drawing:
// * uncomment the statement for a drawing.
// * comment out the statement for a part.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetAttachedEntities.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            object swSelObj1 = null;
            object swSelObj2 = null;
            Annotation swAnn = default(Annotation);
            int[] vAttEntTypeArr = null;
            object[] vAttEntArr = null;
            int i = 0;
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            //Select the note to move
            swSelObj1 = (object)swSelMgr.GetSelectedObject6(1, -1);
            Note swNote = default(Note);
            swNote = (Note)swSelObj1;
            swAnn = (Annotation)swNote.GetAnnotation();

            //Part - select the entity where to move the note
            swSelObj2 = (object)swSelMgr.GetSelectedObject6(2, -1);

            //Drawing - select the entity where to move the note
            //swSelObj2 = (object)swSelMgr.GetSelectedObject6(3, -1);

            object[] AttEntArr = new object[1];
            AttEntArr[0] = swSelObj2;
            DispatcherWrapper[] vAttEntArrIn = null;
            vAttEntArrIn = (DispatcherWrapper[])ObjectArrayToDispatcherWrapperArray(AttEntArr);
            bRet = swAnn.SetAttachedEntities(vAttEntArrIn);
            Debug.Print("Name = " + swAnn.GetName());
            Debug.Print(" Selection Type = " + swSelMgr.GetSelectedObjectType3(1, -1));
            Debug.Print(" Annotation Type = " + swAnn.GetType());
            vAttEntArr = (object[])swAnn.GetAttachedEntities3();
            vAttEntTypeArr = (int[])swAnn.GetAttachedEntityTypes();
            if ((vAttEntTypeArr != null))
            {
                for (i = 0; i < vAttEntTypeArr.Length; i++)
                {
                    //A dangling dimension has at least one entity of type swSelNOTHING
                    Debug.Print(" Entity Type(" + i + ") = " + vAttEntTypeArr[i]);
                    SelectData swSelData = default(SelectData);
                    swSelData = (SelectData)swSelMgr.CreateSelectData();
                    Entity swEntity = default(Entity);
                    swEntity = (Entity)vAttEntArr[i];
                    swEntity.Select4(false, swSelData);
                }
            }
        }

        public DispatcherWrapper[] ObjectArrayToDispatcherWrapperArray(object[] Objects)
        {
            int ArraySize = 0;
            ArraySize = Objects.GetUpperBound(0);
            DispatcherWrapper[] d = new DispatcherWrapper[ArraySize + 1];
            int ArrayIndex = 0;
            for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
            {
                d[ArrayIndex] = new DispatcherWrapper(Objects[ArrayIndex]);
            }
            return d;
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Attach Annotation to Entity Example (VBA)

This example shows how to attach an annotation to a different entity.

```
'-----
' Preconditions:
' 1. Open a part or drawing that has at least one
'    annotation.
' 2. Select an annotation.
' 3. Press Ctrl while selecting a face, edge, or vertex.
'
' Postconditions:
' 1. Attaches the selected annotation to the entity
'    selected in Preconditions step 3, if possible.
' 2. Examine the annotation in the graphics area.
'
' NOTE: If you open a drawing:
' * uncomment the statement for a drawing.
' * comment out the statement for a part.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelObj1 As Object
    Dim swSelObj2 As Object
    Dim swAnn As SldWorks.Annotation
    Dim vAttEntTypeArr As Variant
    Dim vAttEntArr As Variant
    Dim nSelType As Long
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    'Select the annotation to move
    Set swSelObj1 = swSelMgr.GetSelectedObject6(1, -1)
    Set swAnn = swSelObj1.GetAnnotation

    'Part - select the entity where to move the annotation
    Set swSelObj2 = swSelMgr.GetSelectedObject6(2, -1)

    'Drawing - select the entity where to to move the annotation
    'Set swSelObj2 = swSelMgr.GetSelectedObject6(3, -1)

    Dim AttEntArr(0) As Object
    Set AttEntArr(0) = swSelObj2
    Dim vAttEntArrIn As Variant
    vAttEntArrIn = AttEntArr

    bRet = swAnn.SetAttachedEntities(vAttEntArrIn)

    Debug.Print "Name = " & swAnn.GetName
    Debug.Print " Selection Type = " & swSelMgr.GetSelectedObjectType3(1, -1)
    Debug.Print " Annotation Type = " & swAnn.GetType

    vAttEntArr = swAnn.GetAttachedEntities3
    vAttEntTypeArr = swAnn.GetAttachedEntityTypes

    If Not IsEmpty(vAttEntTypeArr) Then
        Debug.Assert UBound(vAttEntArr) = UBound(vAttEntTypeArr)
        For i = 0 To UBound(vAttEntTypeArr)
            'A dangling dimension has at least one entity of type swSelNOTHING
            Debug.Print " Entity Type(" & i & ") = " & vAttEntTypeArr(i)
            Dim swSelData As SldWorks.SelectData
            Set swSelData = swSelMgr.CreateSelectData
            Call vAttEntArr(i).Select4(False, swSelData)
        Next i
    End If
End Sub
```

Attach Annotation to Entity Example (VB.NET)

This example shows how to attach an annotation to a different entity.

```

-----
Preconditions:
1. Open a part or drawing that has at least one
   annotation.
2. Select an annotation.
3. Press Ctrl while selecting a face, edge, or vertex.
Postconditions:
1. Attaches the selected annotation to the entity
   selected in Preconditions step 3, if possible.
2. Examine the annotation in the graphics area.
NOTE: If you open a drawing:
* uncomment the statement for a drawing.
* comment out the statement for a part.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swSelObj1 As Object
        Dim swSelObj2 As Object
        Dim swAnn As Annotation
        Dim vAttEntTypeArr As Object
        Dim vAttEntArr As Object
        Dim i As Integer
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager

        'Select the annotation to move
        swSelObj1 = swSelMgr.GetSelectedObject6(1, -1)
        swAnn = swSelObj1.GetAnnotation

        'Part - select the entity where to move the annotation
        swSelObj2 = swSelMgr.GetSelectedObject6(2, -1)

        'Drawing - select the entity where to to move the annotation
        'swSelObj2 = swSelMgr.GetSelectedObject6(3, -1)

        Dim AttEntArr(0) As Object
        AttEntArr(0) = swSelObj2
        Dim vAttEntArrIn() As DispatcherWrapper
        vAttEntArrIn = ObjectArrayToDispatcherWrapperArray(AttEntArr)
        bRet = swAnn.SetAttachedEntities(vAttEntArrIn)
        Debug.Print("Name = " & swAnn.GetName)
        Debug.Print(" Selection Type = " & swSelMgr.GetSelectedObjectType3(1, -1))
        Debug.Print(" Annotation Type = " & swAnn.GetType)
        vAttEntArr = swAnn.GetAttachedEntities3
        vAttEntTypeArr = swAnn.GetAttachedEntityTypes
        If Not vAttEntTypeArr Is Nothing Then
            Debug.Assert(UBound(vAttEntArr) = UBound(vAttEntTypeArr))
            For i = 0 To UBound(vAttEntTypeArr)
                'A dangling dimension has at least one entity of type swSelNOTHING
                Debug.Print(" Entity Type(" & i & ") = " & vAttEntTypeArr(i))
                Dim swSelData As SelectData
                swSelData = swSelMgr.CreateSelectData
                Call vAttEntArr(i).Select4(False, swSelData)
            Next i
        End If

    End Sub

    Function ObjectArrayToDispatcherWrapperArray(ByVal Objects As Object()) As DispatcherWrapper()
        Dim ArraySize As Integer
        ArraySize = Objects.GetUpperBound(0)
        Dim d(ArraySize) As DispatcherWrapper
        Dim ArrayIndex As Integer
        For ArrayIndex = 0 To ArraySize
            d(ArrayIndex) = New DispatcherWrapper(Objects(ArrayIndex))
        Next
        Return d
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Attach Files to MBD 3D PDF Example (C#)

This example shows how to attach files to a SOLIDWORKS MBD 3D PDF.

```
//-----
// Preconditions:
// 1. Verify that the specified SOLIDWORKS MBD 3D PDF theme,
//    files to attach, and C:\temp exist.
// 2. Open public_documents\samples\tutorial\api\block.sldprt.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets the MBD3DPdfData object.
// 2. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
// 3. Sets the theme for the SOLIDWORKS MBD 3D PDF.
// 4. Sets the standard views for the SOLIDWORKS MBD 3D PDF.
// 5. Sets the level of accuracy for lossy compression, whether to
//    apply lossy compression to polygons in the model, and
//    whether to export SOLIDWORKS Design parts and assemblies with PMI
//    annotations to STEP 242 format.
// 6. Sets the files to attach to SOLIDWORKS MBD 3D PDF.
// 7. Creates the SOLIDWORKS MBD 3D PDF.
// 8. Examine the Immediate window.
// 9. Open C:\temp\block.PDF and click the paper clip (Attachments:
//    View file attachments) in the left-side panel.
//
// NOTE: Because the part is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MBD3DPdfData swMBDPdfData = default(MBD3DPdfData);
            object standardViews = null;
            int[] viewIDs = new int[3];
            string[] filesToAttach = new string[2];
            object attachedFiles = null;
            int status = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Get MBD3DPdfData object
            swMBDPdfData = (MBD3DPdfData)swModelDocExt.GetMBD3DPdfData();

            //Set path and file name for SOLIDWORKS MBD 3D PDF
            swMBDPdfData.FilePath = "C:\\temp\\block.PDF";

            //Set standard views for SOLIDWORKS MBD 3D PDF
            viewIDs[0] = (int)swStandardViews_e.swFrontView;
            viewIDs[1] = (int)swStandardViews_e.swTopView;
            viewIDs[2] = (int)swStandardViews_e.swDimetricView;
            standardViews = viewIDs;
            swMBDPdfData.SetStandardViews(standardViews);

            //Set SOLIDWORKS MBD 3D PDF theme
            swMBDPdfData.ThemeName = "C:\\Program Files\\SolidWorks Corp\\SOLIDWORKS\\data\\themes\\simple part (a4, portrait)\\theme.xml";

            //Set the level of accuracy for lossy compression, whether to apply
            //lossy compression to polygons in the model, and whether to export
            //SOLIDWORKS Design parts and assemblies with PMI annotations to STEP 242
            //format
            swMBDPdfData.Accuracy = (int)sw3DPDFAccuracy_e.swLow;
            swMBDPdfData.CompressLossyTessellation = true;
            swMBDPdfData.CreateAttachSTEP242 = true;

            //Set the files to attach
            filesToAttach[0] = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\2012-sm.slddrw";
            filesToAttach[1] = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\2012-sm.sldprt";
            attachedFiles = filesToAttach;
            swMBDPdfData.SetAttachments(attachedFiles);

            //Publish to PDF
            status = swModelDocExt.PublishTo3DPDF(swMBDPdfData);
            Debug.Print("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " + status);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Attach Files to MBD 3D PDF Example (VBA)

This example shows how to attach files to a SOLIDWORKS MBD 3D PDF.

```
'-----
' Preconditions:
' 1. Verify that the specified SOLIDWORKS MBD 3D PDF theme,
'    files to attach, and C:\temp exist.
' 2. Open public_documents\samples\tutorial\api\block.sldprt.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the MBD3DPdfData object.
' 2. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
' 3. Sets the theme for the SOLIDWORKS MBD 3D PDF.
' 4. Sets the standard views for the SOLIDWORKS MBD 3D PDF.
' 5. Sets the level of accuracy for lossy compression, whether to
'    apply lossy compression to polygons in the model, and
'    whether to export SOLIDWORKS Design parts and assemblies with PMI
'    annotations to STEP 242 format.
' 6. Sets the files to attach to SOLIDWORKS MBD 3D PDF.
' 7. Creates the SOLIDWORKS MBD 3D PDF.
' 8. Examine the Immediate window.
' 9. Open C:\temp\block.PDF and click the paper clip (Attachments:
'    View file attachments) in the left-side panel.
'
' NOTE: Because the part is used elsewhere, do not save
' changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMBDPdfData As SldWorks.MBD3DPdfData
Dim standardViews As Variant
Dim viewIDs(2) As Long
Dim filesToAttach(1) As String
Dim attachedFiles As Variant
Dim status As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    'Get MBD3DPdfData object
    Set swMBDPdfData = swModelDocExt.GetMBD3DPdfData

    'Set path and file name for SOLIDWORKS MBD 3D PDF
    swMBDPdfData.FilePath = "C:\temp\block.PDF"

    'Set standard views for SOLIDWORKS MBD 3D PDF
    viewIDs(0) = swStandardViews_e.swFrontView
    viewIDs(1) = swStandardViews_e.swTopView
    viewIDs(2) = swStandardViews_e.swDimetricView
    standardViews = viewIDs
    swMBDPdfData.SetStandardViews (standardViews)

    'Set SOLIDWORKS MBD 3D PDF theme
    swMBDPdfData.ThemeName = "C:\Program Files\SolidWorks Corp\SOLIDWORKS\data\themes\simple part (a4, portrait)\theme.xml"

    'Set the level of accuracy for lossy compression, whether to apply
    'lossy compression to polygons in the model, and whether to export
    'SOLIDWORKS Design parts and assemblies with PMI annotations to STEP 242
    'format
    swMBDPdfData.Accuracy = sw3DPDFAccuracy_e.swLow
    swMBDPdfData.CompressLossyTessellation = True
    swMBDPdfData.CreateAttachSTEP242 = True

    'Set the files to attach
    filesToAttach(0) = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.slddrw"
    filesToAttach(1) = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt"
    attachedFiles = filesToAttach
    swMBDPdfData.SetAttachments attachedFiles

    'Publish to PDF
    status = swModelDocExt.PublishTo3DPDF(swMBDPdfData)
    Debug.Print ("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " & status)

End Sub
```

Attach Files to MBD 3D PDF Example (VB.NET)

This example shows how to attach files to a SOLIDWORKS MBD 3D PDF.

```

-----
Preconditions:
1. Verify that the specified SOLIDWORKS MBD 3D PDF theme,
   files to attach, and C:\temp exist.
2. Open public_documents\samples\tutorial\api\block.sldprt.
3. Open the Immediate window.

Postconditions:
1. Gets the MBD3DPdfData object.
2. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
3. Sets the theme for the SOLIDWORKS MBD 3D PDF.
4. Sets the standard views for the SOLIDWORKS MBD 3D PDF.
5. Sets the level of accuracy for lossy compression, whether to
   apply lossy compression to polygons in the model, and
   whether to export SOLIDWORKS Design parts and assemblies with PMI
   annotations to STEP 242 format.
6. Sets the files to attach to SOLIDWORKS MBD 3D PDF.
7. Creates the SOLIDWORKS MBD 3D PDF.
8. Examine the Immediate window.
9. Open C:\temp\block.PDF and click the paper clip (Attachments:
   View file attachments) in the left-side panel.

NOTE: Because the part is used elsewhere, do not save
changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swMBDPdfData As MBD3DPdfData
        Dim standardViews As Object
        Dim viewIDs(2) As Integer
        Dim filesToAttach(1) As String
        Dim attachedFiles As Object
        Dim status As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        'Get MBD3DPdfData object
        swMBDPdfData = swModelDocExt.GetMBD3DPdfData

        'Set path and file name for SOLIDWORKS MBD 3D PDF
        swMBDPdfData.FilePath = "C:\temp\block.PDF"

        'Set standard views for SOLIDWORKS MBD 3D PDF
        viewIDs(0) = swStandardViews_e.swFrontView
        viewIDs(1) = swStandardViews_e.swTopView
        viewIDs(2) = swStandardViews_e.swDimetricView
        standardViews = viewIDs
        swMBDPdfData.SetStandardViews(standardViews)

        'Set SOLIDWORKS MBD 3D PDF theme
        swMBDPdfData.ThemeName = "C:\Program Files\SolidWorks Corp\SOLIDWORKS\data\themes\simple part (a4, portrait)\theme.xml"

        'Set the level of accuracy for lossy compression, whether to apply
        'lossy compression to polygons in the model, and whether to export
        'SOLIDWORKS Design parts and assemblies with PMI annotations to STEP 242
        'format
        swMBDPdfData.Accuracy = sw3DPDFAccuracy_e.swLow
        swMBDPdfData.CompressLossyTessellation = True
        swMBDPdfData.CreateAttachSTEP242 = True

        'Set the files to attach
        filesToAttach(0) = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.slddrw"
        filesToAttach(1) = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt"
        attachedFiles = filesToAttach
        swMBDPdfData.SetAttachments(attachedFiles)

        'Publish to PDF
        status = swModelDocExt.PublishTo3DPDF(swMBDPdfData)
        Debug.Print("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Auto-arrange Dimensions Example (C#)

This example shows how to auto-arrange the selected dimensions.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
// 2. Box-select the dimensions in Drawing View1.
//
// Postconditions:
// 1. Auto-arranges the selected dimensions.
// 2. Examine Drawing View1.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

using System;

namespace AlignDimensionsCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DrawingDoc swDrawingDoc = default(DrawingDoc);
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swDrawingDoc = (DrawingDoc)swModel;
            status = swModelDocExt.AlignDimensions((int)swAlignDimensionType_e.swAlignDimensionType_AutoArrange,
0.001);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Auto-arrange Dimensions Example (VBA)

This example shows how to auto-arrange the selected dimensions.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Box-select the dimensions in Drawing View1.
'
' Postconditions:
' 1. Auto-arranges the selected dimensions.
' 2. Examine Drawing View1.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim status As Boolean
Dim errors As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swDrawingDoc = swModel
    status = swModelDocExt.AlignDimensions(swAlignDimensionType_e.swAlignDimensionType_AutoArrange, 0.001)

End Sub
```

Auto-arrange Dimensions Example (VB.NET)

This example shows how to auto-arrange the selected dimensions.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
' 2. Box-select the dimensions in Drawing View1.  
'  
' Postconditions:  
' 1. Auto-arranges the selected dimensions.  
' 2. Examine Drawing View1.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Public Sub Main()  
  
        Dim swModel As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim swDrawingDoc As DrawingDoc  
        Dim status As Boolean  
  
        swModel = swApp.ActiveDoc  
        swModelDocExt = swModel.Extension  
        swDrawingDoc = swModel  
        status = swModelDocExt.AlignDimensions(swAlignDimensionType_e.swAlignDimensionType_AutoArrange, 0.001)  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks  
  
End Class
```

Automatically Insert Center Marks Example (C#)

This example shows how to automatically insert center marks in multiple drawing views.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
//
// Postconditions:
// 1. Clears the Tools > Options > Document Properties > Centerlines/Center Marks >
//    Scale by view scale check box.
// 2. Activates Sheet3.
// 3. Suppresses Drawing View9.
// 4. Inserts center marks in Drawing View9 and Drawing View11.
// 5. Unsuppresses Drawing View9.
// 6. Examine the drawing.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----
using System;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace AutoInsertCenterMarks_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        DrawingDoc Draw;
        ModelDocExtension ModelDocExt;
        View swActiveView;
        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            Draw = (DrawingDoc)Part;
            ModelDocExt = (ModelDocExtension)Part.Extension;

            // Clear the Scale by view scale check box to set gap
            ModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDetailingCenterMarkScaleByViewScale, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);

            Draw.ActivateSheet("Sheet3");

            // Suppress Drawing View9
            boolstatus = ModelDocExt.SelectByID2("Drawing View9", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            Draw.SuppressView();

            // Insert center marks for all holes, fillets, and slots in the specified view
            boolstatus = Draw.ActivateView("Drawing View9");
            swActiveView = (View)Draw.ActiveDrawingView;
            boolstatus = swActiveView.AutoInsertCenterMarks2(7, 11, true, true, true, 0.0025, 0.0025, true, true, 0);

            boolstatus = Draw.ActivateView("Drawing View11");
            swActiveView = (View)Draw.ActiveDrawingView;
            boolstatus = swActiveView.AutoInsertCenterMarks2(7, 11, true, true, false, 0.005, 0.005, true, false, 0);

            Part.ClearSelection2(true);

            // Unsuppress Drawing View9
            boolstatus = ModelDocExt.SelectByID2("Drawing View9", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            Draw.UnsuppressView();
        }
    }

    public SldWorks swApp;
}
}
```

Automatically Insert Center Marks Example (VBA)

This example shows how to automatically insert center marks in multiple drawing views.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
'
' Postconditions:
' 1. Clears the Tools > Options > Document Properties > Centerlines/Center Marks >
'   Scale by view scale check box.
' 2. Activates Sheet3.
' 3. Suppresses Drawing View9.
' 4. Inserts center marks in Drawing View9 and Drawing View11.
' 5. Unsuppresses Drawing View9.
' 6. Examine the drawing.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim ModelDocExt As SldWorks.ModelDocExtension
Dim Draw As SldWorks.DrawingDoc
Dim swActiveView As SldWorks.View
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Set ModelDocExt = Part.Extension
    Set Draw = Part

    ' Clear Scale by view scale check box to set gap
    ModelDocExt.SetUserPreferenceToggle swUserPreferenceToggle_e.swDetailingCenterMarkScaleByViewScale,
    swUserPreferenceOption_e.swDetailingNoOptionSpecified, False

    Draw.ActivateSheet ("Sheet3")

    ' Suppress Drawing View9
    boolstatus = ModelDocExt.SelectByID2("Drawing View9", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Draw.SuppressView

    ' Insert center marks for all holes, fillets, and slots in the specified views
    boolstatus = Draw.ActivateView("Drawing View9")
    Set swActiveView = Draw.ActiveDrawingView
    boolstatus = swActiveView.AutoInsertCenterMarks2(7, _
        11, _
        True, _
        True, _
        True, _
        0.0025, _
        0.0025, _
        True, _
        True, _
        0)

    boolstatus = Draw.ActivateView("Drawing View11")
    Set swActiveView = Draw.ActiveDrawingView
    boolstatus = swActiveView.AutoInsertCenterMarks2(7, _
        11, _
        True, _
        True, _
        False, _
        0.005, _
        0.005, _
        True, _
        False, _
        0)

    Part.ClearSelection2 True

    ' Unsuppress Drawing View9
    boolstatus = ModelDocExt.SelectByID2("Drawing View9", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Draw.UnsuppressView

End Sub

```

Automatically Insert Center Marks Example (VB.NET)

This example shows how to automatically insert center marks in multiple drawing views.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
'
```

```
' Postconditions:
' 1. Clears the Tools > Options > Document Properties > Centerlines/Center Marks >
'    Scale by view scale check box.
' 2. Activates Sheet3.
' 3. Suppresses Drawing View9.
' 4. Inserts center marks in Drawing View9 and Drawing View11.
' 5. Unsuppresses Drawing View9.
' 6. Examine the drawing.
'
```

```
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
```

```
    Dim Draw As DrawingDoc
```

```
    Dim ModelDocExt As ModelDocExtension
```

```
    Dim swActiveView As View
```

```
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
```

```
        Draw = Part
```

```
        ModelDocExt = Part.Extension
```

```
        ' Clear the Scale by view scale check box to set gap
```

```
        ModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDetailingCenterMarkScaleByViewScale, swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
```

```
        Draw.ActivateSheet("Sheet3")
```

```
        ' Suppress Drawing View9
```

```
        boolstatus = ModelDocExt.SelectByID2("Drawing View9", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
```

```
        Draw.SuppressView()
```

```
        ' Insert center marks for all holes, fillets, and slots in the specified views
```

```
        boolstatus = Draw.ActivateView("Drawing View9")
```

```
        swActiveView = Draw.ActiveDrawingView
```

```
        boolstatus = swActiveView.AutoInsertCenterMarks2(7, _
                                                         11, _
                                                         True, _
                                                         True, _
                                                         True, _
                                                         0.0025, _
                                                         0.0025, _
                                                         True, _
                                                         True, _
                                                         0)
```

```
        boolstatus = Draw.ActivateView("Drawing View11")
```

```
        swActiveView = Draw.ActiveDrawingView
```

```
        boolstatus = swActiveView.AutoInsertCenterMarks2(7, _
                                                         11, _
                                                         True, _
                                                         True, _
                                                         False, _
                                                         0.005, _
                                                         0.005, _
                                                         True, _
                                                         False, _
                                                         0)
```

```
        Part.ClearSelection2(True)
```

```
        ' Unsuppress Drawing View9
```

```
        boolstatus = ModelDocExt.SelectByID2("Drawing View9", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        Draw.UnsuppressView()
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Autodimension a Sketch Example (VBA)

This example shows how to autodimension a sketch.

```
'-----
' Preconditions:
' 1. Verify that the specified part document template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Inserts a sketch of a rectangle.
' 3. Selects two sketch segments in the sketch.
'   * First selected sketch segment is used for horizontal datum.
'   * Second selected sketch segment is used for vertical datum.
' 4. Autodimensions the selected sketch segments.
' 5. Examine the Immediate window and graphics area.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSketch As SldWorks.Sketch
    Dim swSketchManager As SldWorks.SketchManager
    Dim sketchLines As Variant
    Dim swSketchSegHoriz As SldWorks.SketchSegment
    Dim swSketchSegVert As SldWorks.SketchSegment
    Dim nRetVal As Long
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension

    bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
    swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
    swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    Set swSketchManager = swModel.SketchManager
    sketchLines = swSketchManager.CreateCornerRectangle(0, 0, 0, 0.110951010058045, -0.066328380491143, 0)
    bRet = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 4.43505736694483E-03, -0.012832795562811, 6.37809258389225E-03, False, 0, Nothing, 0)
    bRet = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0.095835993249203, -3.06185999393385E-02, -2.97695225643872E-02, True, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swSketchSegHoriz = swSelMgr.GetSelectedObject6(1, -1)
    Set swSketchSegVert = swSelMgr.GetSelectedObject6(2, -1)

    swModel.ClearSelection2 True

    ' Reselect sketch segments with correct marks
    ' for autodimensioning
    bRet = swSketchSegHoriz.Select3(True, swAutodimMarkHorizontalDatum, Nothing)
    bRet = swSketchSegVert.Select3(True, swAutodimMarkVerticalDatum, Nothing)

    Set swSketch = swModel.GetActiveSketch2
    nRetVal = swSketch.AutoDimension2(swAutodimEntitiesAll, swAutodimSchemeBaseline, swAutodimHorizontalPlacementBelow, swAutodimSchemeBaseline,
    swAutodimVerticalPlacementLeft)

    Debug.Print "Status of autodimensioning sketch (0 = success): " & nRetVal

    ' Redraw to display dimensions
    swModel.GraphicsRedraw2

End Sub
```

Autodimension All Sketches Example (C#)

This example shows how to autodimension all under-constrained sketches in a part.

```
//-----
// Preconditions:
// 1. Verify that the specified part document template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Inserts two sketches.
// 3. Autodimensions the sketches.
// 4. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace AutoDimension2AllCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        const string swTnProfileFeature = "ProfileFeature";
        const double nTolerance = 1E-08;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            PartDoc swPart = default(PartDoc);
            object[] sketchLines = null;
            string[] sSketchNameArr = null;
            string sSketchName = null;
            Feature swFeat = default(Feature);
            Sketch swSketch = default(Sketch);
            SketchSegment sketchSegment = default(SketchSegment);
            SketchManager swSketchMgr = default(SketchManager);
            int nRetVal = 0;
            bool bRet = false;

            // Create new part document and insert sketches
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            bRet = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);
            bRet = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, true);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            sketchLines = (object[])swSketchMgr.CreateCornerRectangle(0, 0, 0, 0.0711560575730914, -0.0480714437538268, 0);
            swModel.ClearSelection2(true);
            bRet = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);
            bRet = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, true);
            sketchLines = (object[])swSketchMgr.CreateCornerRectangle(-0.100496797175254, 0.0481170506199078, 0, -0.0506046179404507, 0.0100165849869995, 0);
            swModel.ClearSelection2(true);
            sketchSegment = (SketchSegment)swSketchMgr.CreateCircle(-0.06189, -0.041869, 0.0, -0.066641, -0.077213, 0.0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);
            sketchSegment = (SketchSegment)swSketchMgr.CreateCircle(-0.032637, 0.106589, 0, -0.021539, 0.095387, 0);
            swModel.ClearSelection2(true);
            sketchSegment = (SketchSegment)swSketchMgr.CreateCircle(0, 0.083617, 0, 0.01255, 0.067729, 0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);
            swModel.ClearSelection2(true);

            // Find all under-constrained sketches
            swPart = (PartDoc)swModel;
            sSketchNameArr = new string[1];
            swFeat = (Feature)swPart.FirstFeature();
            while ((swFeat != null)) {
                if (swTnProfileFeature == swFeat.GetTypeName2()) {
                    swSketch = (Sketch)swFeat.GetSpecificFeature2();
                    if ((int)swConstrainedStatus_e.swUnderConstrained == swSketch.GetConstrainedStatus()) {
                        sSketchNameArr[sSketchNameArr.GetUpperBound(0)] = (string)swFeat.Name;
                        Array.Resize(ref sSketchNameArr, sSketchNameArr.GetUpperBound(0) + 2);
                    }
                }
                swFeat = (Feature)swFeat.GetNextFeature();
            }

            // Autodimension all under-constrained sketches
            Array.Resize(ref sSketchNameArr, sSketchNameArr.GetUpperBound(0));
            foreach (string sSketchName_loopVariable in sSketchNameArr)
            {
                sSketchName = sSketchName_loopVariable;
                swFeat = (Feature)swPart.FeatureByName(sSketchName);
                swSketch = (Sketch)swFeat.GetSpecificFeature2();
                nRetVal = AutoDimensionSketch(swApp, swModel, swSketch);
                Debug.Print(sSketchName + " autodimensioned (0 = success): " + nRetVal);
            }
        }

        public void FindAllUnderConstrainedSketches(SldWorks swApp, ModelDoc2 swModel, string[] sSketchNameArr)
        {
            PartDoc swPart = default(PartDoc);
            Feature swFeat = default(Feature);
            Sketch swSketch = default(Sketch);
        }
    }
}
```

```

swPart = (PartDoc)swModel;
swFeat = (Feature)swPart.FirstFeature();
while ((swFeat != null))
{
    if (swTnProfileFeature == swFeat.GetTypeName2())
    {
        swSketch = (Sketch)swFeat.GetSpecificFeature2();
        if ((int)swConstrainedStatus_e.swUnderConstrained == swSketch.GetConstrainedStatus())
        {
            sSketchNameArr[sSketchNameArr.GetUpperBound(0)] = (string)swFeat.Name;
            Array.Resize(ref sSketchNameArr, sSketchNameArr.GetUpperBound(0) + 2);
        }
    }
    swFeat = (Feature)swFeat.GetNextFeature();
}
// Remove last empty sketch name
Array.Resize(ref sSketchNameArr, sSketchNameArr.GetUpperBound(0));
}
public object GetAllSketchLines(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch)
{
    object functionReturnValue = null;
    object[] vSketchSegArr = null;
    SketchSegment vSketchSeg = null;
    SketchSegment swSketchSeg = default(SketchSegment);
    SketchLine swSketchCurrLine = default(SketchLine);
    SketchLine[] swSketchLineArr = null;

    swSketchLineArr = new SketchLine[1];
    vSketchSegArr = (object[])swSketch.GetSketchSegments();
    if ((vSketchSegArr != null))
    {
        foreach (SketchSegment vSketchSeg_loopVariable in vSketchSegArr)
        {
            vSketchSeg = (SketchSegment)vSketchSeg_loopVariable;
            swSketchSeg = (SketchSegment)vSketchSeg;
            if ((int)swSketchSegments_e.swSketchLINE == swSketchSeg.GetType())
            {
                swSketchCurrLine = (SketchLine)swSketchSeg;
                swSketchLineArr[swSketchLineArr.GetUpperBound(0)] = (SketchLine)swSketchCurrLine;
                Array.Resize(ref swSketchLineArr, swSketchLineArr.GetUpperBound(0) + 2);
            }
        }
    }
    if (0 == swSketchLineArr.GetUpperBound(0))
    {
        // No straight lines in this sketch
        functionReturnValue = null;
        return functionReturnValue;
    }
    // Remove last empty sketch line
    Array.Resize(ref swSketchLineArr, swSketchLineArr.GetUpperBound(0));
    functionReturnValue = (object[])swSketchLineArr;
    return functionReturnValue;
}

public bool GetSketchPoint(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch, SketchPoint swSketchPt)
{
    bool functionReturnValue = false;
    object[] vSketchPtArr = null;

    vSketchPtArr = (object[])swSketch.GetSketchPoints2();
    if ((vSketchPtArr != null))
    {
        // Use first point
        swSketchPt = (SketchPoint)vSketchPtArr[0];
        functionReturnValue = true;
        return functionReturnValue;
    }
    functionReturnValue = false;
    return functionReturnValue;
}

public bool FindVerticalOrigin(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch, SketchSegment swSketchSegVert, SketchPoint swSketchPtVert)
{
    bool functionReturnValue = false;
    SketchLine[] vSketchLineArr = null;
    SketchLine vSketchLine = null;
    SketchLine swSketchCurrLine = default(SketchLine);
    SketchPoint swStartPt = default(SketchPoint);
    SketchPoint swEndPt = default(SketchPoint);

    // Try to get first vertical line
    vSketchLineArr = (SketchLine[])GetAllSketchLines(swApp, swModel, swSketch);
    if ((vSketchLineArr != null))
    {
        foreach (SketchLine vSketchLine_loopVariable in vSketchLineArr)
        {
            vSketchLine = (SketchLine)vSketchLine_loopVariable;
            swSketchCurrLine = (SketchLine)vSketchLine;
            swStartPt = (SketchPoint)swSketchCurrLine.GetStartPoint2();
            swEndPt = (SketchPoint)swSketchCurrLine.GetEndPoint2();
            if (Math.Abs(swStartPt.X - swEndPt.X) < nTolerance)
            {
                swSketchSegVert = (SketchSegment)swSketchCurrLine;
                functionReturnValue = true;
                return functionReturnValue;
            }
        }
    }
    // Try to get the first point
    functionReturnValue = GetSketchPoint(swApp, swModel, swSketch, swSketchPtVert);
    return functionReturnValue;
}

public bool FindHorizontalOrigin(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch, SketchSegment swSketchSegHoriz, SketchPoint swSketchPtHoriz)
{
    bool functionReturnValue = false;
    SketchLine[] vSketchLineArr = null;
    SketchLine vSketchLine = null;
    SketchLine swSketchCurrLine = default(SketchLine);
    SketchPoint swStartPt = default(SketchPoint);
    SketchPoint swEndPt = default(SketchPoint);

    // Try to get first horizontal line
    vSketchLineArr = (SketchLine[])GetAllSketchLines(swApp, swModel, swSketch);
    if ((vSketchLineArr != null))
    {
        foreach (SketchLine vSketchLine_loopVariable in vSketchLineArr)
        {
            vSketchLine = (SketchLine)vSketchLine_loopVariable;

```

```

        swSketchCurrLine = (SketchLine)vSketchLine;
        swStartPt = (SketchPoint)swSketchCurrLine.GetStartPoint2();
        swEndPt = (SketchPoint)swSketchCurrLine.GetEndPoint2();
        if (Math.Abs(swStartPt.Y - swEndPt.Y) < nTolerance)
        {
            swSketchSegHoriz = (SketchSegment)swSketchCurrLine;
            functionReturnValue = true;
            return functionReturnValue;
        }
    }
    // Try to get the first point
    functionReturnValue = GetSketchPoint(swApp, swModel, swSketch, swSketchPtHoriz);
    return functionReturnValue;
}

public int AutoDimensionSketch(SldWorks swApp, ModelDoc2 swModel, Sketch swSketch)
{
    int functionReturnValue = 0;
    Feature swFeat = default(Feature);
    SketchSegment swSketchSegHoriz = null;
    SketchPoint swSketchPtHoriz = null;
    SketchSegment swSketchSegVert = null;
    SketchPoint swSketchPtVert = null;
    SketchManager swSketchMgr = null;
    bool bRet = false;

    if (false == FindHorizontalOrigin(swApp, swModel, swSketch, swSketchSegHoriz, swSketchPtHoriz))
    {
        functionReturnValue = (int)swAutodimStatus_e.swAutodimStatusDatumLineNotHorizontal;
        return functionReturnValue;
    }
    if (false == FindVerticalOrigin(swApp, swModel, swSketch, swSketchSegVert, swSketchPtVert))
    {
        functionReturnValue = (int)swAutodimStatus_e.swAutodimStatusDatumLineNotVertical;
        return functionReturnValue;
    }
    swFeat = (Feature)swSketch;
    bRet = swFeat.Select2(false, 0);
    // Editing sketch clears selections
    swModel.EditSketch();
    // Reselect sketch segments for autodimensioning
    if ((swSketchSegVert != null))
    {
        // Vertical line is for horizontal datum
        bRet = swSketchSegVert.Select4(true, null);
    }
    else if ((swSketchPtHoriz != null))
    {
        bRet = swSketchPtHoriz.Select4(true, null);
    }
    else if ((swSketchPtVert != null))
    {
        // Use any sketch point for horizontal datum
        bRet = swSketchPtVert.Select4(true, null);
    }
    if ((swSketchSegHoriz != null))
    {
        // Horizontal line is for vertical datum
        bRet = swSketchSegHoriz.Select4(true, null);
    }
    else if ((swSketchPtVert != null))
    {
        bRet = swSketchPtVert.Select4(true, null);
    }
    else if ((swSketchPtHoriz != null))
    {
        // Use any sketch point for vertical datum
        bRet = swSketchPtHoriz.Select4(true, null);
    }
    // No straight lines, probably contains circles,
    // so use sketch points for datums
    if ((GetAllSketchLines(swApp, swModel, swSketch) == null))
    {
        if ((swSketchPtHoriz != null))
        {
            bRet = swSketchPtHoriz.Select4(false, null);
        }
        else if ((swSketchPtVert != null))
        {
            bRet = swSketchPtVert.Select4(false, null);
        }
    }
    functionReturnValue = swSketch.AutoDimension2((int)swAutodimEntities_e.swAutodimEntitiesAll, (int)swAutodimScheme_e.swAutodimSchemeBaseline, (int)swAutodimHorizontalPlacement_e.swAutodimHorizontalPlacementBelow, (int)swAutodimScheme_e.swAutodimSchemeBaseline, (int)swAutodimVerticalPlacement_e.swAutodimVerticalPlacementLeft);
    // Redraw so dimensions are displayed
    swModel.GraphicsRedraw2();
    // Exit sketch mode
    swSketchMgr = swModel.SketchManager;
    swSketchMgr.InsertSketch(false);
    return functionReturnValue;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Autodimension All Sketches Example (VBA)

This example shows how to autodimension all under-constrained sketches in a part.

```
'-----
' Preconditions:
' 1. Verify that the specified part document template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Inserts two sketches.
' 3. Autodimensions the sketches.
' 4. Examine the Immediate window.
'-----
Option Explicit
```

```
Const swTnProfileFeature As String = "ProfileFeature"
Const nTolerance As Double = 0.00000001
```

```
Sub FindAllUnderConstrainedSketches(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, sSketchNameArr() As String)
```

```
    Dim swPart As SldWorks.PartDoc
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch

    Set swPart = swModel
    Set swFeat = swPart.FirstFeature
    Do While Not swFeat Is Nothing
        If swTnProfileFeature = swFeat.GetTypeName2 Then
            Set swSketch = swFeat.GetSpecificFeature2
            If swUnderConstrained = swSketch.GetConstrainedStatus Then
                sSketchNameArr(UBound(sSketchNameArr)) = swFeat.Name
                ReDim Preserve sSketchNameArr(UBound(sSketchNameArr) + 1)
            End If
        End If
        Set swFeat = swFeat.GetNextFeature
    Loop

    ' Remove last empty sketch name
    ReDim Preserve sSketchNameArr(UBound(sSketchNameArr) - 1)
```

```
End Sub
```

```
Function GetAllSketchLines(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch) As Variant
```

```
    Dim vSketchSegArr As Variant
    Dim vSketchSeg As Variant
    Dim swSketchSeg As SldWorks.sketchSegment
    Dim swSketchCurrLine As SldWorks.SketchLine
    Dim swSketchLineArr() As SldWorks.SketchLine
    ReDim swSketchLineArr(0)

    vSketchSegArr = swSketch.GetSketchSegments
    If Not IsEmpty(vSketchSegArr) Then
        For Each vSketchSeg In vSketchSegArr
            Set swSketchSeg = vSketchSeg
            If swSketchLINE = swSketchSeg.GetType Then
                Set swSketchCurrLine = swSketchSeg
                Set swSketchLineArr(UBound(swSketchLineArr)) = swSketchCurrLine
                ReDim Preserve swSketchLineArr(UBound(swSketchLineArr) + 1)
            End If
        Next
    End If

    If 0 = UBound(swSketchLineArr) Then
        ' No straight lines in this sketch
        GetAllSketchLines = Empty
        Exit Function
    End If

    ' Remove last empty sketch line
    ReDim Preserve swSketchLineArr(UBound(swSketchLineArr) - 1)
    GetAllSketchLines = swSketchLineArr
```

```
End Function
```

```
Function GetSketchPoint(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSketchPt As SldWorks.SketchPoint) As Boolean
```

```
    Dim vSketchPtArr As Variant

    vSketchPtArr = swSketch.GetSketchPoints2
    If Not IsEmpty(vSketchPtArr) Then
        ' Use first point
        Set swSketchPt = vSketchPtArr(0)
        GetSketchPoint = True
        Exit Function
    End If

    GetSketchPoint = False
```

```
End Function
```

```
Function FindVerticalOrigin(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSketchSegVert As SldWorks.sketchSegment, swSketchPtVert As SldWorks.SketchPoint) As Boolean
```

```

Dim vSketchLineArr As Variant
Dim vSketchLine As Variant
Dim swSketchCurrLine As SldWorks.SketchLine
Dim swStartPt As SldWorks.SketchPoint
Dim swEndPt As SldWorks.SketchPoint

' Try to get first vertical line
vSketchLineArr = GetAllSketchLines(swApp, swModel, swSketch)

If Not IsEmpty(vSketchLineArr) Then
    For Each vSketchLine In vSketchLineArr
        Set swSketchCurrLine = vSketchLine
        Set swStartPt = swSketchCurrLine.GetStartPoint2
        Set swEndPt = swSketchCurrLine.GetEndPoint2

        If Abs(swStartPt.X - swEndPt.X) < nTolerance Then
            Set swSketchSegVert = swSketchCurrLine
            FindVerticalOrigin = True
            Exit Function
        End If
    Next
End If

' Try to get the first point
FindVerticalOrigin = GetSketchPoint(swApp, swModel, swSketch, swSketchPtVert)

End Function

Function FindHorizontalOrigin(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch, swSketchSegHoriz As
SldWorks.sketchSegment, swSketchPtHoriz As SldWorks.SketchPoint) As Boolean

    Dim vSketchLineArr As Variant
    Dim vSketchLine As Variant
    Dim swSketchCurrLine As SldWorks.SketchLine
    Dim swStartPt As SldWorks.SketchPoint
    Dim swEndPt As SldWorks.SketchPoint

    ' Try to get first horizontal line
    vSketchLineArr = GetAllSketchLines(swApp, swModel, swSketch)

    If Not IsEmpty(vSketchLineArr) Then
        For Each vSketchLine In vSketchLineArr
            Set swSketchCurrLine = vSketchLine
            Set swStartPt = swSketchCurrLine.GetStartPoint2
            Set swEndPt = swSketchCurrLine.GetEndPoint2

            If Abs(swStartPt.Y - swEndPt.Y) < nTolerance Then
                Set swSketchSegHoriz = swSketchCurrLine
                FindHorizontalOrigin = True
                Exit Function
            End If
        Next
    End If

    ' Try to get the first point
    FindHorizontalOrigin = GetSketchPoint(swApp, swModel, swSketch, swSketchPtHoriz)

End Function

Function AutoDimensionSketch(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSketch As SldWorks.Sketch) As Long

    Dim swFeat As SldWorks.Feature
    Dim swSketchSegHoriz As SldWorks.sketchSegment
    Dim swSketchPtHoriz As SldWorks.SketchPoint
    Dim swSketchSegVert As SldWorks.sketchSegment
    Dim swSketchPtVert As SldWorks.SketchPoint
    Dim swSketchMgr as SldWorks.SketchManager
    Dim bRet As Boolean

    If False = FindHorizontalOrigin(swApp, swModel, swSketch, swSketchSegHoriz, swSketchPtHoriz) Then
        AutoDimensionSketch = swAutodimStatusDatumLineNotHorizontal
        Exit Function
    End If

    If False = FindVerticalOrigin(swApp, swModel, swSketch, swSketchSegVert, swSketchPtVert) Then
        AutoDimensionSketch = swAutodimStatusDatumLineNotVertical
        Exit Function
    End If

    Set swFeat = swSketch
    bRet = swFeat.Select2(False, 0)

    ' Editing sketch clears selections
    swModel.EditSketch

    ' Reselect sketch segments for autodimensioning
    If Not swSketchSegVert Is Nothing Then
        ' Vertical line is for horizontal datum
        bRet = swSketchSegVert.Select4(True, Nothing)
    ElseIf Not swSketchPtHoriz Is Nothing Then
        bRet = swSketchPtHoriz.Select4(True, Nothing)
    ElseIf Not swSketchPtVert Is Nothing Then
        ' Use any sketch point for horizontal datum
        bRet = swSketchPtVert.Select4(True, Nothing)
    End If

    If Not swSketchSegHoriz Is Nothing Then
        ' Horizontal line is for vertical datum
        bRet = swSketchSegHoriz.Select4(True, Nothing)
    ElseIf Not swSketchPtVert Is Nothing Then
        bRet = swSketchPtVert.Select4(True, Nothing)
    ElseIf Not swSketchPtHoriz Is Nothing Then
        ' Use any sketch point for vertical datum
        bRet = swSketchPtHoriz.Select4(True, Nothing)
    End If

```

```

' No straight lines, probably contains circles,
' so use sketch points for datums
If IsEmpty(GetAllSketchLines(swApp, swModel, swSketch)) Then
    If Not swSketchPtHoriz Is Nothing Then
        bRet = swSketchPtHoriz.Select4(False, Nothing)
    ElseIf Not swSketchPtVert Is Nothing Then
        bRet = swSketchPtVert.Select4(False, Nothing)
    End If
End If

AutoDimensionSketch = swSketch.AutoDimension2(swAutodimEntitiesAll, swAutodimSchemeBaseline, swAutodimHorizontalPlacementBelow,
swAutodimSchemeBaseline, swAutodimVerticalPlacementLeft)

' Redraw so dimensions are displayed
swModel.GraphicsRedraw2

' Exit sketch mode
Set swSketchMgr = swModel.SketchManager
swSketchMgr.InsertSketch False

End Function

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swPart As SldWorks.PartDoc
    Dim sketchLines As Variant
    Dim sSketchNameArr() As String
    Dim sSketchName As Variant
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim sketchSegment As SldWorks.sketchSegment
    Dim swSketchMgr As SldWorks.SketchManager
    Dim nRetVal As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")

    ' Create new part document and sketch
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    Set swSketchMgr = swModel.SketchManager
    sketchLines = swSketchMgr.CreateCornerRectangle(0, 0, 0, 7.11560575730914E-02, -4.80714437538268E-02, 0)
    swModel.ClearSelection2 True
    bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    sketchLines = swSketchMgr.CreateCornerRectangle(-0.100496797175254, 4.81170506199078E-02, 0, -5.06046179404507E-02, 1.00165849869995E-02, 0)
    swModel.ClearSelection2 True
    Set sketchSegment = swSketchMgr.CreateCircle(-0.06189, -0.041869, 0#, -0.066641, -0.077213, 0#)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True
    Set sketchSegment = swSketchMgr.CreateCircle(-0.032637, 0.106589, 0#, -0.021539, 0.095387, 0#)
    swModel.ClearSelection2 (True)
    Set sketchSegment = swSketchMgr.CreateCircle(0#, 0.083617, 0#, 0.01255, 0.067729, 0#)
    swModel.ClearSelection2 (True)
    swSketchMgr.InsertSketch (True)
    swModel.ClearSelection2 (True)

    Set swPart = swModel

    ReDim sSketchNameArr(0)

    ' Find all under-constrained sketches
    FindAllUnderConstrainedSketches swApp, swModel, sSketchNameArr

    ' Autodimension all under-constrained sketches
    For Each sSketchName In sSketchNameArr
        Set swFeat = swPart.FeatureByName(sSketchName)
        Set swSketch = swFeat.GetSpecificFeature2

        nRetVal = AutoDimensionSketch(swApp, swModel, swSketch)

        Debug.Print sSketchName & " autodimensioned (0 = success): " & nRetVal
    Next

End Sub

```

Autodimension All Sketches Example (VB.NET)

This example shows how to autodimension all under-constrained sketches in a part.

```

-----
Preconditions:
1. Verify that the specified part document template exists.
2. Open the Immediate window.
-----
Postconditions:
1. Opens a new part document.
2. Inserts two sketches.
3. Autodimensions the sketches.
4. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro
    Const swTnProfileFeature As String = "ProfileFeature"
    Const nTolerance As Double = 0.00000001

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swPart As PartDoc
        Dim sketchLines As Object
        Dim sSketchNameArr() As String
        Dim sSketchName As Object
        Dim swFeat As Feature
        Dim swSketch As Sketch
        Dim sketchSegment As SketchSegment
        Dim swSketchMgr As SketchManager
        Dim nRetVal As Integer
        Dim bRet As Boolean

        swApp = CreateObject("SldWorks.Application")

        ' Create new part document and inserts sketches
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prt", 0, 0, 0)
        swModelDocExt = swModel.Extension
        bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOptionSp
ecified, False)
        bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOpti
onSpecified, True)
        swSketchMgr = swModel.SketchManager
        sketchLines = swSketchMgr.CreateCornerRectangle(0, 0, 0, 0.0711560575730914, -0.0480714437538268, 0)
        swModel.ClearSelection2(True)
        bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOptionSp
ecified, False)
        bRet = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOpti
onSpecified, True)
        sketchLines = swSketchMgr.CreateCornerRectangle(-0.100496797175254, 0.0481170506199078, 0, -0.0506046179404507, 0.0100165849869995, 0)
        swModel.ClearSelection2(True)
        sketchSegment = swSketchMgr.CreateCircle(-0.06189, -0.041869, 0.0#, -0.066641, -0.077213, 0.0#)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)
        sketchSegment = swSketchMgr.CreateCircle(-0.032637, 0.106589, 0.0#, -0.021539, 0.095387, 0.0#)
        swModel.ClearSelection2(True)
        sketchSegment = swSketchMgr.CreateCircle(0.0#, 0.083617, 0.0#, 0.01255, 0.067729, 0.0#)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)
        swModel.ClearSelection2(True)

        ' Find all under-constrained sketches
        ReDim sSketchNameArr(0)
        swPart = swModel
        swFeat = swPart.FirstFeature
        Do While Not swFeat Is Nothing
            If swTnProfileFeature = swFeat.GetTypeName2 Then
                swSketch = swFeat.GetSpecificFeature2
                If swConstrainedStatus_e.swUnderConstrained = swSketch.GetConstrainedStatus Then
                    sSketchNameArr(UBound(sSketchNameArr)) = swFeat.Name
                    ReDim Preserve sSketchNameArr(UBound(sSketchNameArr) + 1)
                End If
            End If
            swFeat = swFeat.GetNextFeature
        Loop

        ' Remove last empty sketch name
        ReDim Preserve sSketchNameArr(UBound(sSketchNameArr) - 1)

        'Autodimension under-constrained sketches
        For Each sSketchName In sSketchNameArr
            swFeat = swPart.FeatureByName(sSketchName)
            swSketch = swFeat.GetSpecificFeature2
            nRetVal = AutoDimensionSketch(swApp, swModel, swSketch)
            Debug.Print(sSketchName & " autodimensioned (0 = success): " & nRetVal)
        Next

    End Sub

    Function GetAllSketchLines(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch) As Object
        Dim vSketchSegArr As Object
        Dim vSketchSeg As Object
        Dim swSketchSeg As SketchSegment
        Dim swSketchCurrLine As SketchLine
        Dim swSketchLineArr() As SketchLine

        ReDim swSketchLineArr(0)

```

```

vSketchSegArr = swSketch.GetSketchSegments
If Not IsNothing(vSketchSegArr) Then
    For Each vSketchSeg In vSketchSegArr
        swSketchSeg = vSketchSeg
        If swSketchSegments_e.swSketchLINE = swSketchSeg.GetType Then
            swSketchCurrLine = swSketchSeg
            swSketchLineArr(UBound(swSketchLineArr)) = swSketchCurrLine
            ReDim Preserve swSketchLineArr(UBound(swSketchLineArr) + 1)
        End If
    Next
End If
If 0 = UBound(swSketchLineArr) Then
    ' No straight lines in this sketch
    GetAllSketchLines = Nothing
    Exit Function
End If
' Remove last empty sketch line
ReDim Preserve swSketchLineArr(UBound(swSketchLineArr) - 1)
GetAllSketchLines = swSketchLineArr
End Function

Function GetSketchPoint(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch, ByVal swSketchPt As SketchPoint) As Boolean
    Dim vSketchPtArr As Object

    vSketchPtArr = swSketch.GetSketchPoints2
    If Not IsNothing(vSketchPtArr) Then
        ' Use first point
        swSketchPt = vSketchPtArr(0)
        GetSketchPoint = True
        Exit Function
    End If
    GetSketchPoint = False
End Function

Function FindVerticalOrigin(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch, ByVal swSketchSegVert As SketchSegment, ByV
al swSketchPtVert As SketchPoint) As Boolean
    Dim vSketchLineArr As Object
    Dim vSketchLine As Object
    Dim swSketchCurrLine As SketchLine
    Dim swStartPt As SketchPoint
    Dim swEndPt As SketchPoint

    ' Try to get first vertical line
    vSketchLineArr = GetAllSketchLines(swApp, swModel, swSketch)
    If Not IsNothing(vSketchLineArr) Then
        For Each vSketchLine In vSketchLineArr
            swSketchCurrLine = vSketchLine
            swStartPt = swSketchCurrLine.GetStartPoint2
            swEndPt = swSketchCurrLine.GetEndPoint2
            If Math.Abs(swStartPt.X - swEndPt.X) < nTolerance Then
                swSketchSegVert = swSketchCurrLine
                FindVerticalOrigin = True
                Exit Function
            End If
        Next
    End If
    ' Try to get the first point
    FindVerticalOrigin = GetSketchPoint(swApp, swModel, swSketch, swSketchPtVert)
End Function

Function FindHorizontalOrigin(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch, ByVal swSketchSegHoriz As SketchSegment,
ByVal swSketchPtHoriz As SketchPoint) As Boolean
    Dim vSketchLineArr As Object
    Dim vSketchLine As Object
    Dim swSketchCurrLine As SketchLine
    Dim swStartPt As SketchPoint
    Dim swEndPt As SketchPoint

    ' Try to get first horizontal line
    vSketchLineArr = GetAllSketchLines(swApp, swModel, swSketch)
    If Not IsNothing(vSketchLineArr) Then
        For Each vSketchLine In vSketchLineArr
            swSketchCurrLine = vSketchLine
            swStartPt = swSketchCurrLine.GetStartPoint2
            swEndPt = swSketchCurrLine.GetEndPoint2
            If Math.Abs(swStartPt.Y - swEndPt.Y) < nTolerance Then
                swSketchSegHoriz = swSketchCurrLine
                FindHorizontalOrigin = True
                Exit Function
            End If
        Next
    End If
    ' Try to get the first point
    FindHorizontalOrigin = GetSketchPoint(swApp, swModel, swSketch, swSketchPtHoriz)
End Function

Function AutoDimensionSketch(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSketch As Sketch) As Integer
    Dim swFeat As Feature
    Dim swSketchSegHoriz As SketchSegment = Nothing
    Dim swSketchPtHoriz As SketchPoint = Nothing
    Dim swSketchSegVert As SketchSegment = Nothing
    Dim swSketchPtVert As SketchPoint = Nothing
    Dim bRet As Boolean
    Dim swSketchMgr As SketchManager = Nothing

    If False = FindHorizontalOrigin(swApp, swModel, swSketch, swSketchSegHoriz, swSketchPtHoriz) Then
        AutoDimensionSketch = swAutodimStatus_e.swAutodimStatusDatumLineNotHorizontal
        Exit Function
    End If
    If False = FindVerticalOrigin(swApp, swModel, swSketch, swSketchSegVert, swSketchPtVert) Then
        AutoDimensionSketch = swAutodimStatus_e.swAutodimStatusDatumLineNotVertical
        Exit Function
    End If
    swFeat = swSketch
    bRet = swFeat.Select2(False, 0)
    ' Editing sketch clears selections
    swModel.EditSketch()
    ' Reselect sketch segments for autodimensioning
    If Not swSketchSegVert Is Nothing Then
        ' Vertical line is for horizontal datum
        bRet = swSketchSegVert.Select4(True, Nothing)
    ElseIf Not swSketchPtHoriz Is Nothing Then
        bRet = swSketchPtHoriz.Select4(True, Nothing)
    ElseIf Not swSketchPtVert Is Nothing Then
        ' Use any sketch point for horizontal datum
        bRet = swSketchPtVert.Select4(True, Nothing)
    End If
    If Not swSketchSegHoriz Is Nothing Then
        ' Horizontal line is for vertical datum
        bRet = swSketchSegHoriz.Select4(True, Nothing)
    End If

```

```

ElseIf Not swSketchPtVert Is Nothing Then
    bRet = swSketchPtVert.Select4(True, Nothing)
ElseIf Not swSketchPtHoriz Is Nothing Then
    ' Use any sketch point for vertical datum
    bRet = swSketchPtHoriz.Select4(True, Nothing)
End If
' No straight lines, probably contains circles,
' so use sketch points for datums
If IsNothing(GetAllSketchLines(swApp, swModel, swSketch)) Then
    If Not swSketchPtHoriz Is Nothing Then
        bRet = swSketchPtHoriz.Select4(False, Nothing)
    ElseIf Not swSketchPtVert Is Nothing Then
        bRet = swSketchPtVert.Select4(False, Nothing)
    End If
End If
AutoDimensionSketch = swSketch.AutoDimension2(swAutodimEntities_e.swAutodimEntitiesAll, swAutodimScheme_e.swAutodimSchemeBaseline, swAutodimHorizontalPlacement_e.swAutodimHorizontalPlacementBelow, swAutodimScheme_e.swAutodimSchemeBaseline, swAutodimVerticalPlacement_e.swAutodimVerticalPlacementLeft)
' Redraw so dimensions are displayed
swModel.GraphicsRedraw2()
' Exit sketch mode
swSketchMgr = swModel.SketchManager
swSketchMgr.InsertSketch(False)
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Autodimension Selected Drawing View Example (C#)

This example shows how to autodimension a selected drawing view.

```
//-----
// Preconditions: Verify that the specified drawing document to
// open exists.
//
// Postconditions:
// 1. Opens the specified drawing document.
// 2. Activates Drawing View1.
// 3. Selects a vertex.
// 4. Autodimensions the drawing view based on the
// selected vertex.
// 5. Examine the drawing.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace AutodimensionCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDrawing = default(DrawingDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            bool status = false;
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            int selmark = 0;
            int ret = 0;

            // Open drawing document of part
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref er
rors, ref warnings);
            swDrawing = (DrawingDoc)swModel;
            status = swDrawing.ActivateView("Drawing View1");
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Select drawing view
            status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);

            // Horizontal and vertical datum, or a vertex datum, baselines for
            // dimension creation
            // These are optional; if not selected, autodimension uses default datums,
            // the leftmost and bottommost edges
            selmark = (int)swAutodimMark_e.swAutodimMarkHorizontalDatum;
            selmark = (int)swAutodimMark_e.swAutodimMarkVerticalDatum;
            selmark = (int)swAutodimMark_e.swAutodimMarkOriginDatum;

            // Select a vertex
            status = swModelDocExt.SelectByID2("", "VERTEX", 0.20215546544586, 0.2496899375, 0.00479999999998881, true, selmark, null, 0);

            // Autodimensions the drawing view based on the selected vertex
            ret = swDrawing.AutoDimension((int)swAutodimEntities_e.swAutodimEntitiesBasedOnPreselect, (int)swAutodimScheme_e.swAutodimSchemeBaseline, (int)
swAutodimHorizontalPlacement_e.swAutodimHorizontalPlacementAbove, (int)swAutodimScheme_e.swAutodimSchemeBaseline, (int)swAutodimVerticalPlacement_e.swAut
odimVerticalPlacementRight);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Autodimension Selected Drawing View Example (VBA)

This example shows how to autodimension a selected drawing view.

```
'-----
' Preconditions: Verify that the specified drawing document to
' open exists.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Activates Drawing View1.
' 3. Selects a vertex.
' 4. Autodimensions the drawing view based on the
'    selected vertex.
' 5. Examine the drawing.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim status As Boolean
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim selmark As Long
Dim ret As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Open drawing document of part
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawing = swModel
    status = swDrawing.ActivateView("Drawing View1")
    Set swModelDocExt = swModel.Extension
    ' Select drawing view
    status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)

    ' Horizontal and vertical datum, or a vertex datum, baselines for
    ' dimension creation
    ' These are optional; if not selected, autodimension uses default datums,
    ' the leftmost and bottommost edges
    selmark = swAutodimMark_e.swAutodimMarkHorizontalDatum
    selmark = swAutodimMark_e.swAutodimMarkVerticalDatum
    selmark = swAutodimMark_e.swAutodimMarkOriginDatum

    ' Select a vertex
    status = swModelDocExt.SelectByID2("", "VERTEX", 0.20215546544586, 0.2496899375, 4.79999999998881E-03, True, selmark, Nothing, 0)

    ' Autodimensions the drawing view based on the selected vertex
    ret = swDrawing.AutoDimension(swAutodimEntities_e.swAutodimEntitiesBasedOnPreselect, swAutodimScheme_e.swAutodimSchemeBaseline,
    swAutodimHorizontalPlacement_e.swAutodimHorizontalPlacementAbove, swAutodimScheme_e.swAutodimSchemeBaseline,
    swAutodimVerticalPlacement_e.swAutodimVerticalPlacementRight)

End Sub
```

Autodimension Selected Drawing View Example (VB.NET)

This example shows how to autodimension a selected drawing view.

```

-----
' Preconditions: Verify that the specified drawing document to
' open exists.
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Activates Drawing View1.
' 3. Selects a vertex.
' 4. Autodimensions the drawing view based on the
'    selected vertex.
' 5. Examine the drawing.
' NOTE: Because the drawing is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swDrawing As DrawingDoc
        Dim swModelDocExt As ModelDocExtension
        Dim status As Boolean
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim selmark As Integer
        Dim ret As Integer

        ' Open drawing document of part
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swDrawing = swModel
        status = swDrawing.ActivateView("Drawing View1")
        swModelDocExt = swModel.Extension

        ' Select drawing view
        status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)

        ' Horizontal and vertical datum, or a vertex datum, baselines for
        ' dimension creation
        ' These are optional; if not selected, autodimension uses default datums,
        ' the leftmost and bottommost edges
        selmark = swAutodimMark_e.swAutodimMarkHorizontalDatum
        selmark = swAutodimMark_e.swAutodimMarkVerticalDatum
        selmark = swAutodimMark_e.swAutodimMarkOriginDatum

        ' Select a vertex
        status = swModelDocExt.SelectByID2("", "VERTEX", 0.20215546544586, 0.2496899375, 0.0047999999999998881, True, selmark, Nothing, 0)

        ' Autodimensions the drawing view based on the selected vertex
        ret = swDrawing.AutoDimension(swAutodimEntities_e.swAutodimEntitiesBasedOnPreselect, swAutodimScheme_e.swAutodimSchemeBaseline, swAutodimHorizontalPlacement_e.swAutodimHorizontalPlacementAbove, swAutodimScheme_e.swAutodimSchemeBaseline, swAutodimVerticalPlacement_e.swAutodimVerticalPlacementRight)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Automate Add-in's Undo Commands Example (VBA)

This example shows how to automate an add-in's undo commands.

- Module
 - [Macro1](#)
- Class modules
 - [swUndoApiHdlr1](#)
 - [swUndoApiHdlr2](#)

```
Module macro1
'-----
'
' Preconditions: Model document is open.
'
' Postconditions: None
'
'-----
Option Explicit

Dim swApp As Object
Dim swModel As SldWorks.ModelDoc2
Dim swModExt As SldWorks.ModelDocExtension
Dim Retval As Boolean
Dim name As String
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModExt = swModel.Extension()

    For i = 1 To 2
        Dim swUndoObj1 As swUndoApiHdlr1
        Set swUndoObj1 = New swUndoApiHdlr1
        swUndoObj1.a = i
        name = "AddinUndo : " & i
        Retval = swModExt.SetApiUndoObject(swUndoObj1, name)

    Next i

    For i = 1 To 2
        Dim swUndoObj2 As swUndoApiHdlr2
        Set swUndoObj2 = New swUndoApiHdlr2
        swUndoObj2.a = i
        name = "AddinUndoReset : " & i
        Retval = swModExt.SetApiUndoObject(swUndoObj2, name)
    Next i

End Sub
```

[Back to top](#)

```
Class module swUndoApiHdlr1
Option Explicit

Implements SwUndoAPIHandler

Public a As Long

Private Sub SwUndoAPIHandler_DoUndo()
    MsgBox "Add-in's undo called " & vbCrLf & "Undo object count : " + Str(a)
End Sub

Private Sub SwUndoAPIHandler_UndoReset()
    MsgBox ("Add-in's undo list reset")
End Sub
```

[Back to top](#)

Class module swUndoApiHdr2

Option Explicit

Implements SwUndoAPIHandler

Public a As Long

Private Sub **SwUndoAPIHandler_DoUndo()**

 MsgBox "Undo called " & vbCrLf & "Undo count : " + Str(a)

End Sub

Private Sub **SwUndoAPIHandler_UndoReSet()**

 MsgBox ("Undo reset called ")

End Sub

[Back to top](#)

Automatically Solved Equations in Sequence Example (C#)

This example shows how to automatically sequence the equations in an order determined to produce accurate results.

```
//-----
// Preconditions:
// 1. Verify that the specified part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Sets the equation's Automatic solve order option
//    to true, which results in the model's
//    independent equations executing before
//    dependent equations, if any.
// 2. Examine the Immediate window.
//
// NOTE: Because this model is used elsewhere,
// do save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace AutomaticSolveOrderCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            EquationMgr swEquationMgr = default(EquationMgr);
            string fileName = null;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\partEquations.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swEquationMgr = (EquationMgr)swModel.GetEquationMgr();
            //Make sure that this model includes equations
            //by getting the number of equations
            Debug.Print("Number of equations: " + swEquationMgr.GetCount());
            if (swEquationMgr.GetCount() >= 1)
            {
                //Get whether equations are automatically
                //solved in sequence
                Debug.Print(" Are equations automatically solved in sequence? " + swEquationMgr.AutomaticSolveOrder);
                if (swEquationMgr.AutomaticSolveOrder)
                {
                    return;
                }
                else
                {
                    //Automatically solve equations in sequence
                    swEquationMgr.AutomaticSolveOrder = true;
                    Debug.Print(" Are equations automatically solved in sequence? " + swEquationMgr.AutomaticSolveOrder);
                }
            }
            else
            {
                Debug.Print("No equations included with this model.");
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Automatically Solve Equations in Sequence Example (VBA)

This example shows how to automatically sequence the equations in an order determined to produce accurate results.

```

'-----
' Preconditions:
' 1. Verify that the specified part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Sets the equation's Automatic solve order option
'    to true, which results in the model's
'    independent equations executing before
'    dependent equations, if any.
' 2. Examine the Immediate window.
'
' NOTE: Because this model is used elsewhere,
' do save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swEquationMgr As SldWorks.EquationMgr
Dim fileName As String
Dim errors As Long, warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\partEquations.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swEquationMgr = swModel.GetEquationMgr
    'Make sure that this model includes equations
    'by getting the number of equations
    Debug.Print "Number of equations: " & swEquationMgr.GetCount
    If swEquationMgr.GetCount >= 1 Then
        'Get whether equations are automatically
        'solved in sequence
        Debug.Print " Are equations automatically solved in sequence? " & swEquationMgr.AutomaticSolveOrder
        If swEquationMgr.AutomaticSolveOrder Then
            Exit Sub
        Else
            'Automatically solve equations in sequence
            swEquationMgr.AutomaticSolveOrder = True
            Debug.Print " Are equations automatically solved in sequence? " & swEquationMgr.AutomaticSolveOrder
        End If
    Else
        Debug.Print "No equations included with this model."
    End If
End Sub

```

Automatically Solve Equations in Sequence Example (VB.NET)

This example shows how to automatically sequence the equations in an order determined to produce accurate results.

```

-----
Preconditions:
1. Verify that the specified part to open exists.
2. Open the Immediate window.
Postconditions:
1. Sets the equation's Automatic solve order option
   to true, which results in the model's
   independent equations executing before
   dependent equations, if any.
2. Examine the Immediate window.
NOTE: Because this model is used elsewhere,
do save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swEquationMgr As EquationMgr
        Dim fileName As String
        Dim errors As Integer, warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\partEquations.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swEquationMgr = swModel.GetEquationMgr
        'Make sure that this model includes equations
        'by getting the number of equations
        Debug.Print("Number of equations: " & swEquationMgr.GetCount)
        If swEquationMgr.GetCount >= 1 Then
            'Get whether equations are automatically
            'solved in sequence
            Debug.Print(" Are equations automatically solved in sequence? " & swEquationMgrAutomaticSolveOrder)
            If swEquationMgrAutomaticSolveOrder Then
                Exit Sub
            Else
                'Automatically solve equations in sequence
                swEquationMgrAutomaticSolveOrder = True
                Debug.Print(" Are equations automatically solved in sequence? " & swEquationMgrAutomaticSolveOrder)
            End If
        Else
            Debug.Print("No equations included with this model.")
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Box Select a Sketch Example (C#)

This example shows how to box select all of the entities in a sketch block.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\blocks\piston_mechanism.sldblk.
//
// Postconditions:
// 1. Opens and box selects Sketch1.
// 2. Examine the list of selected entities
//    in the PropertyManager page.
// 3. Interactively quit editing the sketch without
//    saving any changes.
// 4. Close the document.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace SketchBoxSelect_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            bool boolstatus = false;
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            // Select the sketch and open it
            boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swModel.EditSketch();
            swModel.ViewZoomtofit2();

            // Box select the sketch
            boolstatus = swModelDocExt.SketchBoxSelect("-0.663893", "1.322001", "0.000000", "-0.395073", "0.698568", "0.000000");
        }
        public SldWorks swApp;
    }
}
```

Box Select a Sketch Example (VBA)

This example shows how to box select all of the entities in a sketch block.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\blocks\piston_mechanism.sldblk.  
'  
' Postconditions:  
' 1. Opens and box selects Sketch1.  
' 2. Examine the list of selected entities  
'    in the PropertyManager page.  
' 3. Interactively quit editing the sketch without  
'    saving any changes.  
' 4. Close the document.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
  
    ' Select the sketch and open it  
    boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)  
    swModel.EditSketch  
    swModel.ViewZoomtofit2  
  
    ' Box select the sketch  
    boolstatus = swModelDocExt.SketchBoxSelect("-0.663893", "1.322001", "0.000000", "-0.395073", "0.698568", "0.000000")  
  
End Sub
```

Box Select a Sketch Example (VB.NET)

This example shows how to box select all of the entities in a sketch block.

```
-----
' Preconditions: Open public_documents\samples\tutorial\blocks\piston_mechanism.sldblk.
'
' Postconditions:
' 1. Opens and box selects Sketch1.
' 2. Examine the list of selected entities
'    in the PropertyManager page.
' 3. Interactively quit editing the sketch without
'    saving any changes.
' 4. Close the document.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacros

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Select the sketch and open it
        boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditSketch()
        swModel.ViewZoomtofit2()

        ' Box select the sketch
        boolstatus = swModelDocExt.SketchBoxSelect("-0.663893", "1.322001", "0.000000", "-0.395073", "0.698568", "0.000000")

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Calculate Closest Distance Between Faces Example (VBA)

This example shows how to calculate the closest distance between two faces.

NOTE: The SOLIDWORKS Design user interface can measure the distance between two selected faces. The corresponding method for this functionality is `IModelDoc2::ClosestDistance`. If any of the faces is cylindrical, then the measurement point is taken from the axis of the cylinder. This is by design and the intended behavior. However, in some circumstances, this measurement might not be appropriate. For example, the minimum amount of material between two holes is normally measured between the cylindrical faces and not between the axes. This sample code shows how to detect such situations and to calculate the distance between faces.

```

'-----
' Preconditions:
' 1. Open a part or fully resolved assembly.
' 2. Select two faces that do not intersect.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Inserts a 3D sketch using the results from
'   the user-interface Measure tool.
' 2. Inserts a 3D sketch showing closest distance
'   between faces.
' 3. Examine the Immediate window, FeatureManager
'   tree, and graphics area.
'-----
Option Explicit

Sub CreateLine(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vPt1 As Variant, vPt2 As Variant)
    Dim swSketchSeg As SldWorks.SketchSegment
    Dim bRet As Boolean

    swModel.SetAddToDB True
    swModel.Insert3DSketch2 False
    Set swSketchSeg = swModel.CreateLine2(vPt1(0), vPt1(1), vPt1(2), vPt2(0), vPt2(1), vPt2(2))
    swModel.SetAddToDB False
    swModel.Insert3DSketch2 True
    bRet = swModel.EditRebuild3
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFace1 As SldWorks.Face2
    Dim swFace2 As SldWorks.Face2
    Dim swSurf1 As SldWorks.Surface
    Dim swSurf2 As SldWorks.Surface
    Dim vPoint1 As Variant
    Dim vPoint2 As Variant
    Dim vClosestPt1 As Variant
    Dim vClosestPt2 As Variant
    Dim nDist As Double

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFace1 = swSelMgr.GetSelectedObject6(1, -1)
    Set swFace2 = swSelMgr.GetSelectedObject6(2, -1)
    Set swSurf1 = swFace1.GetSurface
    Set swSurf2 = swFace2.GetSurface

    'Get the result of the measure command
    'Need the points returned
    nDist = swModel.ClosestDistance(swFace1, swFace2, vPoint1, vPoint2)
    Debug.Print " ClosestDistance = " & nDist * 1000# & " mm"

    CreateLine swApp, swModel, vPoint1, vPoint2

    'Use the points returned by the user-interface Measure tool
    'to get the nearest point actually on the faces in question.
    If swSurf1.IsCylinder Then
        'Measure has returned the center point,
        'so use the point on the other surface
        vClosestPt1 = swFace1.GetClosestPointOn(vPoint2(0), vPoint2(1), vPoint2(2))
    Else
        'Probably on the surface, but just to be sure
        vClosestPt1 = swFace1.GetClosestPointOn(vPoint1(0), vPoint1(1), vPoint1(2))
    End If

    If swSurf2.IsCylinder Then
        'Measure tool returned the center point, so use
        'the point on the other surface
        vClosestPt2 = swFace2.GetClosestPointOn(vPoint1(0), vPoint1(1), vPoint1(2))
    Else
        'Probably on the surface, but just to be sure
        vClosestPt2 = swFace2.GetClosestPointOn(vPoint2(0), vPoint2(1), vPoint2(2))
    End If

    nDist = Sqr((vClosestPt1(0) - vClosestPt2(0)) ^ 2 + (vClosestPt1(1) - vClosestPt2(1)) ^ 2 + (vClosestPt1(2) - vClosestPt2(2)) ^ 2)
    Debug.Print " Distance = " & nDist * 1000# & " mm"

    CreateLine swApp, swModel, vClosestPt1, vClosestPt2
End Sub

```

Calculate Closest Distance Between Model Components Example (VBA)

This example shows how to calculate the closest distance between two components in an assembly.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\landing_gear.sldasm.
' 2. Click a face on oleostrut<1>.
' 3. Ctrl+click a face on lwrsway_lnk<1>.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Creates a 3D sketch using the selection points and calculates
'    the distance between the components.
' 2. Examine the Immediate window, graphics area, and
'    the FeatureManager design tree.
'
' NOTE: Because this assembly is used elsewhere, do not save
' changes.
'-----
```

Option Explicit

```
Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFace1 As SldWorks.Face2
    Dim swFace2 As SldWorks.Face2
    Dim vPt1 As Variant
    Dim vPt2 As Variant
    Dim nDist As Double
    Dim swSkPoint1 As SldWorks.SketchPoint
    Dim swSkPoint2 As SldWorks.SketchPoint
    Dim swSkLine As SldWorks.SketchLine
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFace1 = swSelMgr.GetSelectedObject6(1, -1)
    Set swFace2 = swSelMgr.GetSelectedObject6(2, -1)

    nDist = swModel.ClosestDistance(swFace1, swFace2, vPt1, vPt2)

    Debug.Assert nDist > 0#
    Debug.Assert Not IsEmpty(vPt1)
    Debug.Assert Not IsEmpty(vPt2)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Selection type (swSelFaces = 2)"
    Debug.Print " 1 = " & swSelMgr.GetSelectedObjectType2(1)
    Debug.Print " 2 = " & swSelMgr.GetSelectedObjectType2(2)
    Debug.Print " Coordinates of selection point"
    Debug.Print " 1 = (" & vPt1(0) * 1000# & ", " & vPt1(1) * 1000# & ", " & vPt1(2) * 1000# & ") mm"
    Debug.Print " 2 = (" & vPt2(0) * 1000# & ", " & vPt2(1) * 1000# & ", " & vPt2(2) * 1000# & ") mm"
    Debug.Print " Distance between selection points = " & nDist * 1000# & " mm"

    swModel.SetAddToDB True

    swModel.Insert3DSketch2 False

    Set swSkPoint1 = swModel.CreatePoint2(vPt1(0), vPt1(1), vPt1(2))
    Set swSkPoint2 = swModel.CreatePoint2(vPt2(0), vPt2(1), vPt2(2))
    Set swSkLine = swModel.CreateLine2(vPt1(0), vPt1(1), vPt1(2), vPt2(0), vPt2(1), vPt2(2))

    swModel.SetAddToDB False

    swModel.Insert3DSketch2 True

End Sub
```

Calculate Environmental Impact of an Assembly Example (C#)

This example shows how to calculate the environmental impact of an assembly.

```
//-----
// Preconditions:
// 1. Add the SOLIDWORKS Sustainability primary interop assembly as
//    a reference (in the IDE's Project Explorer, right-click
//    the project name, select Add Reference, click the Browse tab,
//    navigate to the install_dir\api\redist folder and
//    select SolidWorks.Interop.sustainability.dll).
// 2. Open:
//    public_documents\samples\tutorial\api\ZZM7005.sldasm
// 3. Open an Immediate window.
// 4. Run to the breakpoint and wait for the environmental impact results to
//    finish updating in the Sustainability Task Pane.
// 5. Press F5 to finish.
//
// Postconditions: Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.sustainability;
using System.Runtime.InteropServices;
namespace SustAssem_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 asmDoc;
        swgsustainabilityApp sustain;
        SustainabilityMaterial materialInfo;
        SustainabilityAssemblyProcess assemblyProcess;
        SustainabilityAssemblyUse assemblyUse;
        SustainabilityTransportation transInfo;
        SustainabilityEndOfLife eolInfo;

        SustainabilityEnvironmentalImpact environmentalImpactInfo;
        object componentList;
        string[] arrayComponent = new string[3];
        object missingComponentList;
        string materialclass;
        string materialname;
        bool assemblyEnergy;
        object assemblyProcessAmt;
        double[] arrayComponent1 = new double[2];
        double BuildToLast;
        bool energylifespan;
        object assemblyUseAmt;
        double[] arrayComponent2 = new double[6];
        double[] CarbonContent;
        int CarbonValues;
        double[] WaterContent;
        int WaterValues;
        double[] AirContent;
        int AirValues;
        double[] EnergyContent;
        int EnergyValues;
        object output;

        public void Main()
        {
            asmDoc = (ModelDoc2)swApp.ActiveDoc;

            sustain = (swgsustainabilityApp)asmDoc.Extension.GetSustainability();

            // Assign material to assembly components
            Debug.Print("Assign material to assembly components:");
            materialInfo = (SustainabilityMaterial)sustain.GetSustainabilityMaterial();

            arrayComponent[0] = "ZZM7005 End Plate (Default)";
            arrayComponent[1] = "ZZM7005 Silencer (04)";
            arrayComponent[2] = "ZZM7005 Silencer (-)";
            componentList = arrayComponent;

            materialInfo.ExcludeComponent(ref componentList);

            Debug.Print(" Number of components missing materials: " + materialInfo.GetMissingMaterialComponentCount());
        }
    }
}
```

```

materialInfo.IncludeComponent(ref componentList);

missingComponentList = materialInfo.GetMissingMaterialComponentList();

materialInfo.SetPropertiesForComponent(ref missingComponentList);

materialclass = "Steel";
materialInfo.MaterialClass = materialclass;
Debug.Print(" Material class: " + materialclass);

materialname = "AISI 1020";
materialInfo.MaterialName = materialname;
Debug.Print(" Material name: " + materialname);

materialInfo.ViewResults();

// Assembly Process
Debug.Print("Assembly Process:");
assemblyProcess = sustain.GetSustainabilityAssemblyProcess();

assemblyProcess.EnergyForAssemblyProcess = true;
assemblyEnergy = assemblyProcess.EnergyForAssemblyProcess;

arrayComponent1[0] = 10.0;
arrayComponent1[1] = 20.0;
assemblyProcessAmt = arrayComponent1;
assemblyProcess.AssemblyProcessAmount = assemblyProcessAmt;

Debug.Print(" Region of assembly as defined in swSustainabilityRegionName_e: " + assemblyProcess.Region);

BuildToLast = 21.0;
assemblyProcess.BuiltToLast = BuildToLast;
Debug.Print(" Built to last: " + assemblyProcess.BuiltToLast);
Debug.Print(" Units of time as defined in swSustainabilityDurationType_e: " + assemblyProcess.DurationType);

// Assembly Use
Debug.Print("Assembly Use:");
assemblyUse = sustain.GetSustainabilityAssemblyUse();

assemblyUse.EnergyOverLifeSpan = true;
energylifespan = assemblyUse.EnergyOverLifeSpan;

arrayComponent2[0] = 10.0;
arrayComponent2[1] = 20.0;
arrayComponent2[2] = 30.0;
arrayComponent2[3] = 40.0;
arrayComponent2[4] = 50.0;
arrayComponent2[5] = 60.0;
assemblyUseAmt = arrayComponent2;
assemblyUse.AssemblyUseAmount = assemblyUseAmt;
Debug.Print(" Per units of time as defined in swSustainabilityDurationType_e: " + assemblyUse.DurationType);
Debug.Print(" Assembly use region as defined in swSustainabilityRegionName_e: " + assemblyUse.AssemblyRegion);

// Transportation
Debug.Print("Transportation (in km):");
transInfo = sustain.GetSustainabilityTransportation();

Debug.Print(" By train: " + transInfo.Train);
Debug.Print(" By truck: " + transInfo.Truck);
Debug.Print(" By boat: " + transInfo.Boat);
Debug.Print(" By plane: " + transInfo.Plane);

// End of Life
Debug.Print("Disposal at end of life:");
eolInfo = sustain.GetSustainabilityEndOfLife();

Debug.Print(" % recycled: " + eolInfo.Recycled);
Debug.Print(" % incinerated: " + eolInfo.Incinerated);
Debug.Print(" % landfill: " + eolInfo.Landfill);

// Environmental Impact
Debug.Print("");
Debug.Print("Calculated Environmental Impact:");

environmentalImpactInfo = sustain.GetSustainabilityEnvironmentalImpact();

Debug.Print(" Time over which to evaluate environmental impact: " + environmentalImpactInfo.DurationOfUse);
Debug.Print(" Units of time over which to evaluate as defined in swSustainabilityDurationType_e: " +
environmentalImpactInfo.DurationType);

environmentalImpactInfo.UpdateResults();

System.Diagnostics.Debugger.Break();

CarbonValues = environmentalImpactInfo.GetCurrentCarbonFootPrint(out output);
CarbonContent = (Double[])output;
Debug.Print(" Carbon footprint");
Debug.Print(" Material: " + CarbonContent[0]);
Debug.Print(" Use: " + CarbonContent[1]);
Debug.Print(" Transportation: " + CarbonContent[2]);
Debug.Print(" Manufacturing: " + CarbonContent[3]);
Debug.Print(" End of Life: " + CarbonContent[4]);

WaterValues = environmentalImpactInfo.GetWaterEutrophication(out output);
WaterContent = (Double[])output;
Debug.Print(" Water eutrophication");

```

```

Debug.Print("  Material: " + WaterContent[0]);
Debug.Print("  Use: " + WaterContent[1]);
Debug.Print("  Transportation: " + WaterContent[2]);
Debug.Print("  Manufacturing: " + WaterContent[3]);
Debug.Print("  End of Life: " + WaterContent[4]);

AirValues = environmentalImpactInfo.GetAirAcidification(out output);
AirContent = (Double[])output;
Debug.Print("  Air acidification");
Debug.Print("  Material: " + AirContent[0]);
Debug.Print("  Use: " + AirContent[1]);
Debug.Print("  Transportation: " + AirContent[2]);
Debug.Print("  Manufacturing: " + AirContent[3]);
Debug.Print("  End of Life: " + AirContent[4]);

EnergyValues = environmentalImpactInfo.GetEnergyConsumption(out output);
EnergyContent = (Double[])output;
Debug.Print("  Energy consumption");
Debug.Print("  Material: " + EnergyContent[0]);
Debug.Print("  Use: " + EnergyContent[1]);
Debug.Print("  Transportation: " + EnergyContent[2]);
Debug.Print("  Manufacturing: " + EnergyContent[3]);
Debug.Print("  End of Life: " + EnergyContent[4]);

```

```

}

```

```

public SldWorks swApp;

```

```

}
}

```

Calculate Environmental Impact of an Assembly Example (VBA)

This example shows how to calculate the environmental impact of an assembly.

```

'-----
' Preconditions:
' 1. Open:
'   public_documents\samples\tutorial\api\ZMZ7005.sldasm
' 2. Open an Immediate window.
' 3. Run to the breakpoint and wait for the environmental impact results to
'   finish updating in the Sustainability Task Pane.
' 4. Press F5 to finish.
'
' Postconditions: Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----

```

```

Dim swApp As SldWorks.SldWorks
Dim asmDoc As SldWorks.ModelDoc2
Dim sustain As sustainabilityLib.swgsustainabilityApp
Dim materialInfo As sustainabilityLib.SustainabilityMaterial
Dim assemblyProcess As sustainabilityLib.SustainabilityAssemblyProcess
Dim assemblyUse As sustainabilityLib.SustainabilityAssemblyUse
Dim transInfo As sustainabilityLib.SustainabilityTransportation
Dim eolInfo As sustainabilityLib.SustainabilityEndOfLife
Dim environmentalImpactInfo As sustainabilityLib.SustainabilityEnvironmentalImpact

```

```

Dim componentList As Variant
Dim arrayComponent(2) As String
Dim missingComponentList As Variant
Dim materialclass As String
Dim materialname As String
Dim assemblyEnergy As Boolean
Dim assemblyProcessAmt As Variant
Dim arrayComponent1(1) As Double
Dim BuildToLast As Double
Dim energylifespan As Boolean
Dim assemblyUseAmt As Variant
Dim arrayComponent2(5) As Double
Dim CarbonContent As Variant
Dim CarbonValues As Long
Dim WaterContent As Variant
Dim WaterValues As Long
Dim AirContent As Variant
Dim AirValues As Long
Dim EnergyContent As Variant
Dim EnergyValues As Long

```

Option Explicit

Sub main()

Set swApp = Application.SldWorks

Set asmDoc = swApp.ActiveDoc

Set sustain = asmDoc.Extension.GetSustainability

```

' Assign material to assembly components
Debug.Print "Assign material to assembly components:"
Set materialInfo = sustain.GetSustainabilityMaterial

```

```

arrayComponent(0) = "ZMZ7005 End Plate (Default)"
arrayComponent(1) = "ZMZ7005 Silencer (04)"
arrayComponent(2) = "ZMZ7005 Silencer (-)"
componentList = arrayComponent

```

materialInfo.ExcludeComponent (arrayComponent(0))

Debug.Print " Number of components missing materials: " & materialInfo.GetMissingMaterialComponentCount

materialInfo.IncludeComponent (arrayComponent(0))

missingComponentList = materialInfo.GetMissingMaterialComponentList

materialInfo.SetPropertiesForComponent (missingComponentList)

```

materialclass = "Steel"
materialInfo.materialclass = materialclass
Debug.Print " Material class: " & materialclass

```

```

materialname = "AISI 1020"
materialInfo.materialname = materialname

```

```

Debug.Print " Material name: " & materialname

materialInfo.ViewResults

' Assembly Process
Debug.Print "Assembly Process:"
Set assemblyProcess = sustain.GetSustainabilityAssemblyProcess

assemblyProcess.EnergyForAssemblyProcess = True
assemblyEnergy = assemblyProcess.EnergyForAssemblyProcess

arrayComponent1(0) = 10#
arrayComponent1(1) = 20#
assemblyProcessAmt = arrayComponent1
assemblyProcess.AssemblyProcessAmount = assemblyProcessAmt

Debug.Print " Region of assembly as defined in swSustainabilityRegionName_e: " & assemblyProcess.region

BuildToLast = 21#
assemblyProcess.BuiltToLast = BuildToLast
Debug.Print " Built to last: " & assemblyProcess.BuiltToLast
Debug.Print " Units of time as defined in swSustainabilityDurationType_e: " & assemblyProcess.durationType

' Assembly Use
Debug.Print "Assembly Use:"
Set assemblyUse = sustain.GetSustainabilityAssemblyUse

assemblyUse.energyOverLifeSpan = True
energylifespan = assemblyUse.energyOverLifeSpan

arrayComponent2(0) = 10#
arrayComponent2(1) = 20#
arrayComponent2(2) = 30#
arrayComponent2(3) = 40#
arrayComponent2(4) = 50#
arrayComponent2(5) = 60#
assemblyUseAmt = arrayComponent2
assemblyUse.AssemblyUseAmount = assemblyUseAmt
Debug.Print " Per units of time as defined in swSustainabilityDurationType_e: " & assemblyUse.durationType
Debug.Print " Assembly use region as defined in swSustainabilityRegionName_e: " & assemblyUse.AssemblyRegion

' Transportation
Debug.Print "Transportation (in km):"
Set transInfo = sustain.GetSustainabilityTransportation

Debug.Print " By train: " & transInfo.Train
Debug.Print " By truck: " & transInfo.Truck
Debug.Print " By boat: " & transInfo.Boat
Debug.Print " By plane: " & transInfo.Plane

' End of Life
Debug.Print "Disposal at end of life:"
Set eolInfo = sustain.GetSustainabilityEndOfLife

Debug.Print " % recycled: " & eolInfo.recycled
Debug.Print " % incinerated: " & eolInfo.incinerated
Debug.Print " % landfill: " & eolInfo.landfill

' Environmental Impact
Debug.Print ""
Debug.Print "Calculated Environmental Impact:"

Set environmentalImpactInfo = sustain.GetSustainabilityEnvironmentalImpact

Debug.Print " Time over which to evaluate environmental impact: " & environmentalImpactInfo.durationofuse
Debug.Print " Units of time over which to evaluate as defined in swSustainabilityDurationType_e: " &
environmentalImpactInfo.durationType

environmentalImpactInfo.UpdateResults

Stop

CarbonValues = environmentalImpactInfo.GetCurrentCarbonFootPrint(CarbonContent)
Debug.Print " Carbon footprint"
Debug.Print " Material: " & CarbonContent(0)
Debug.Print " Use: " & CarbonContent(1)
Debug.Print " Transportation: " & CarbonContent(2)
Debug.Print " Manufacturing: " & CarbonContent(3)
Debug.Print " End of Life: " & CarbonContent(4)

WaterValues = environmentalImpactInfo.GetWaterEutrophication(WaterContent)
Debug.Print " Water eutrophication"
Debug.Print " Material: " & WaterContent(0)
Debug.Print " Use: " & WaterContent(1)
Debug.Print " Transportation: " & WaterContent(2)

```

```
Debug.Print " Manufacturing: " & WaterContent(3)
Debug.Print " End of Life: " & WaterContent(4)

AirValues = environmentalImpactInfo.GetAirAcidification(AirContent)
Debug.Print " Air acidification"
Debug.Print " Material: " & AirContent(0)
Debug.Print " Use: " & AirContent(1)
Debug.Print " Transportation: " & AirContent(2)
Debug.Print " Manufacturing: " & AirContent(3)
Debug.Print " End of Life: " & AirContent(4)

EnergyValues = environmentalImpactInfo.GetEnergyConsumption(EnergyContent)
Debug.Print " Energy consumption"
Debug.Print " Material: " & EnergyContent(0)
Debug.Print " Use: " & EnergyContent(1)
Debug.Print " Transportation: " & EnergyContent(2)
Debug.Print " Manufacturing: " & EnergyContent(3)
Debug.Print " End of Life: " & EnergyContent(4)
```

End Sub

Calculate Environmental Impact of an Assembly Example (VB.NET)

This example shows how to calculate the environmental impact of an assembly.

```

'-----
' Preconditions:
' 1. Add the SOLIDWORKS Sustainability primary interop assembly as
'    a reference (in the IDE's Project Explorer, right-click
'    the project name, select Add Reference, click the Browse tab,
'    navigate to the install_dir\api\redist folder and
'    select SolidWorks.Interop.sustainability.dll).
' 2. Open:
'    public_documents\samples\tutorial\api\ZZM7005.sldasm
' 3. Open an Immediate window.
' 4. Run to the breakpoint and wait for the environmental impact results to
'    finish updating in the Sustainability Task Pane.
' 5. Press F5 to finish.
'
' Postconditions: Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.sustainability
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim asmDoc As ModelDoc2
    Dim sustain As swgsustainabilityApp
    Dim materialInfo As SustainabilityMaterial
    Dim assemblyProcess As SustainabilityAssemblyProcess
    Dim assemblyUse As SustainabilityAssemblyUse
    Dim transInfo As SustainabilityTransportation
    Dim eolInfo As SustainabilityEndOfLife
    Dim environmentalImpactInfo As SustainabilityEnvironmentalImpact

```

```

    Dim componentList As Object
    Dim arrayComponent(2) As String
    Dim missingComponentList As Object
    Dim materialclass As String
    Dim materialname As String
    Dim assemblyEnergy As Boolean
    Dim assemblyProcessAmt As Object
    Dim arrayComponent1(1) As Double
    Dim BuildToLast As Double
    Dim energylifespan As Boolean
    Dim assemblyUseAmt As Object
    Dim arrayComponent2(5) As Double
    Dim CarbonContent As Object
    Dim CarbonValues As Integer
    Dim WaterContent As Object
    Dim WaterValues As Integer
    Dim AirContent As Object
    Dim AirValues As Integer
    Dim EnergyContent As Object
    Dim EnergyValues As Integer

```

```
Sub main()
```

```
    asmDoc = swApp.ActiveDoc
```

```
    sustain = asmDoc.Extension.GetSustainability
```

```

    ' Assign material to assembly components
    Debug.Print("Assign material to assembly components:")
    materialInfo = sustain.GetSustainabilityMaterial

```

```

    arrayComponent(0) = "ZZM7005 End Plate (Default)"
    arrayComponent(1) = "ZZM7005 Silencer (04)"
    arrayComponent(2) = "ZZM7005 Silencer (-)"
    componentList = arrayComponent

```

```
    materialInfo.ExcludeComponent(arrayComponent(0).ToString())
```

```
    Debug.Print("  Number of components missing materials: " & materialInfo.GetMissingMaterialComponentCount)
```

```
    materialInfo.IncludeComponent(arrayComponent(0).ToString())
```

```
    missingComponentList = materialInfo.GetMissingMaterialComponentList
```

```
    materialInfo.SetPropertiesForComponent(missingComponentList)
```

```
    materialclass = "Steel"
```

```

materialInfo.materialclass = materialclass
Debug.Print(" Material class: " & materialclass)

materialname = "AISI 1020"
materialInfo.materialname = materialname
Debug.Print(" Material name: " & materialname)

materialInfo.ViewResults()

' Assembly Process
Debug.Print("Assembly Process:")
assemblyProcess = sustain.GetSustainabilityAssemblyProcess

assemblyProcess.EnergyForAssemblyProcess = True
assemblyEnergy = assemblyProcess.EnergyForAssemblyProcess

arrayComponent1(0) = 10.0#
arrayComponent1(1) = 20.0#
assemblyProcessAmt = arrayComponent1
assemblyProcess.AssemblyProcessAmount = assemblyProcessAmt

Debug.Print(" Region of assembly as defined in swSustainabilityRegionName_e: " & assemblyProcess.region)

BuildToLast = 21.0#
assemblyProcess.BuiltToLast = BuildToLast
Debug.Print(" Built to last: " & assemblyProcess.BuiltToLast)
Debug.Print(" Units of time as defined in swSustainabilityDurationType_e: " & assemblyProcess.durationType)

' Assembly Use
Debug.Print("Assembly Use:")
assemblyUse = sustain.GetSustainabilityAssemblyUse

assemblyUse.energyOverLifeSpan = True
energylifespan = assemblyUse.energyOverLifeSpan

arrayComponent2(0) = 10.0#
arrayComponent2(1) = 20.0#
arrayComponent2(2) = 30.0#
arrayComponent2(3) = 40.0#
arrayComponent2(4) = 50.0#
arrayComponent2(5) = 60.0#
assemblyUseAmt = arrayComponent2
assemblyUse.AssemblyUseAmount = assemblyUseAmt
Debug.Print(" Per units of time as defined in swSustainabilityDurationType_e: " & assemblyUse.durationType)
Debug.Print(" Assembly use region as defined in swSustainabilityRegionName_e: " & assemblyUse.AssemblyRegion)

' Transportation
Debug.Print("Transportation (in km):")
transInfo = sustain.GetSustainabilityTransportation

Debug.Print(" By train: " & transInfo.Train)
Debug.Print(" By truck: " & transInfo.Truck)
Debug.Print(" By boat: " & transInfo.Boat)
Debug.Print(" By plane: " & transInfo.Plane)

' End of Life
Debug.Print("Disposal at end of life:")
eolInfo = sustain.GetSustainabilityEndOfLife

Debug.Print(" % recycled: " & eolInfo.recycled)
Debug.Print(" % incinerated: " & eolInfo.incinerated)
Debug.Print(" % landfill: " & eolInfo.landfill)

' Environmental Impact
Debug.Print("")
Debug.Print("Calculated Environmental Impact:")

environmentalImpactInfo = sustain.GetSustainabilityEnvironmentalImpact

Debug.Print(" Time over which to evaluate environmental impact: " & environmentalImpactInfo.durationofuse)
Debug.Print(" Units of time over which to evaluate as defined in swSustainabilityDurationType_e: " &
environmentalImpactInfo.durationType)

environmentalImpactInfo.UpdateResults()

Stop

CarbonValues = environmentalImpactInfo.GetCurrentCarbonFootPrint(CarbonContent)
Debug.Print(" Carbon footprint")
Debug.Print(" Material: " & CarbonContent(0))
Debug.Print(" Use: " & CarbonContent(1))
Debug.Print(" Transportation: " & CarbonContent(2))
Debug.Print(" Manufacturing: " & CarbonContent(3))
Debug.Print(" End of Life: " & CarbonContent(4))

WaterValues = environmentalImpactInfo.GetWaterEutrophication(WaterContent)
Debug.Print(" Water eutrophication")
Debug.Print(" Material: " & WaterContent(0))
Debug.Print(" Use: " & WaterContent(1))
Debug.Print(" Transportation: " & WaterContent(2))
Debug.Print(" Manufacturing: " & WaterContent(3))
Debug.Print(" End of Life: " & WaterContent(4))

AirValues = environmentalImpactInfo.GetAirAcidification(AirContent)
Debug.Print(" Air acidification")
Debug.Print(" Material: " & AirContent(0))

```

```
Debug.Print("  Use: " & AirContent(1))
Debug.Print("  Transportation: " & AirContent(2))
Debug.Print("  Manufacturing: " & AirContent(3))
Debug.Print("  End of Life: " & AirContent(4))

EnergyValues = environmentalImpactInfo.GetEnergyConsumption(EnergyContent)
Debug.Print("  Energy consumption")
Debug.Print("  Material: " & EnergyContent(0))
Debug.Print("  Use: " & EnergyContent(1))
Debug.Print("  Transportation: " & EnergyContent(2))
Debug.Print("  Manufacturing: " & EnergyContent(3))
Debug.Print("  End of Life: " & EnergyContent(4))
```

End Sub

Public swApp As SldWorks

End Class

Calculate Environmental Impact of Part (C#)

This example shows how to calculate the environmental impact of a part.

```
//-----
// Preconditions:
// 1. Add the SOLIDWORKS Sustainability primary interop assembly as
//    a reference (in the IDE's Project Explorer, right-click
//    the project name, select Add Reference, click the Browse tab,
//    navigate to the install_dir\api\redist folder and
//    select SolidWorks.Interop.sustainability.dll).
// 2. Open:
//    public_documents\samples\tutorial\api\ZZM7005 End Plate.sldprt
// 3. Open an Immediate window.
// 4. Run to the breakpoint and wait for the environmental impact results to
//    finish updating in the Sustainability Task Pane.
// 5. Press F5 to finish.
//
// Postconditions: Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.sustainability;
using System.Runtime.InteropServices;
namespace SustPart_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 PDoc = default(ModelDoc2);
            PDoc = (ModelDoc2)swApp.ActiveDoc;

            swgsustainabilityApp sustain = default(swgsustainabilityApp);
            sustain = (swgsustainabilityApp)PDoc.Extension.GetSustainability();

            // Material
            Debug.Print("Material:");
            SustainabilityMaterial materialInfo = default(SustainabilityMaterial);
            materialInfo = sustain.GetSustainabilityMaterial();

            string MaterialClass = null;
            MaterialClass = "Plastics";
            materialInfo.MaterialClass = MaterialClass;
            Debug.Print(" Material class: " + MaterialClass);

            string MaterialName = null;
            MaterialName = "ABS";
            materialInfo.MaterialName = MaterialName;
            Debug.Print(" Material name: " + MaterialName);

            Debug.Print(" % recycled: " + materialInfo.RecycledContent);
            Debug.Print(" Weight: " + materialInfo.Weight);

            // Manufacturing
            Debug.Print("Manufacturing:");
            SustainabilityManufacturing manufacturingInfo = default(SustainabilityManufacturing);
            manufacturingInfo = sustain.GetSustainabilityManufacturing();

            Debug.Print(" Region: " + manufacturingInfo.ManufacturingRegion);
            Debug.Print(" Built to last: " + manufacturingInfo.BuiltToLast);
            Debug.Print(" Process: " + manufacturingInfo.ManufacturingProcess);
            Debug.Print(" Primary electricity: " + manufacturingInfo.PrimaryElectricity);
            Debug.Print(" Primary natural gas: " + manufacturingInfo.PrimaryNaturalGas);
            Debug.Print(" Primary scrap rate: " + manufacturingInfo.PrimaryScrapRate);
            Debug.Print(" Paint: " + manufacturingInfo.Paint);
            Debug.Print(" Surface area: " + manufacturingInfo.SurfaceArea);

            // Transportation
            Debug.Print("Transportation:");
            SustainabilityTransportation transInfo = default(SustainabilityTransportation);
            transInfo = sustain.GetSustainabilityTransportation();

            Debug.Print(" By train: " + transInfo.Train);
            Debug.Print(" By truck: " + transInfo.Truck);
            Debug.Print(" By boat: " + transInfo.Boat);
            Debug.Print(" By plane: " + transInfo.Plane);

            // Disposal at end of life
        }
    }
}
```

```

Debug.Print("Disposal at end of life:");
SustainabilityEndOfLife eofInfo = default(SustainabilityEndOfLife);
eofInfo = sustain.GetSustainabilityEndOfLife();

Debug.Print(" % recycled: " + eofInfo.Recycled);
Debug.Print(" % incinerated: " + eofInfo.Incinerated);
Debug.Print(" % landfill: " + eofInfo.Landfill);

// Use
Debug.Print("Use:");
SustainabilityPartUse partInfo = default(SustainabilityPartUse);
partInfo = sustain.GetSustainabilityPartUse();

Debug.Print(" Region of use: " + partInfo.Region);

System.Diagnostics.Debugger.Break();

// Environmental impact
Debug.Print("Environmental impact:");
SustainabilityEnvironmentalImpact environmentalImpactInfo = default(SustainabilityEnvironmentalImpact);
environmentalImpactInfo = sustain.GetSustainabilityEnvironmentalImpact();

int CarbonValues = 0;
object output = null;
Double[] CarbonContent;
CarbonValues = environmentalImpactInfo.GetCurrentCarbonFootPrint(out output);
CarbonContent = (Double[])output;
Debug.Print(" Carbon footprint");
Debug.Print(" Material: " + CarbonContent[0]);
Debug.Print(" Use: " + CarbonContent[1]);
Debug.Print(" Transportation: " + CarbonContent[2]);
Debug.Print(" Manufacturing: " + CarbonContent[3]);
Debug.Print(" End of Life: " + CarbonContent[4]);

int WaterValues = 0;
output = null;
Double[] WaterContent;
WaterValues = environmentalImpactInfo.GetWaterEutrophication(out output);
WaterContent = (Double[])output;
Debug.Print(" Water eutrophication");
Debug.Print(" Material: " + WaterContent[0]);
Debug.Print(" Use: " + WaterContent[1]);
Debug.Print(" Transportation: " + WaterContent[2]);
Debug.Print(" Manufacturing: " + WaterContent[3]);
Debug.Print(" End of Life: " + WaterContent[4]);

int AirValues = 0;
output = null;
Double[] AirContent;
AirValues = environmentalImpactInfo.GetAirAcidification(out output);
AirContent = (Double[])output;
Debug.Print(" Air acidification");
Debug.Print(" Material: " + AirContent[0]);
Debug.Print(" Use: " + AirContent[1]);
Debug.Print(" Transportation: " + AirContent[2]);
Debug.Print(" Manufacturing: " + AirContent[3]);
Debug.Print(" End of Life: " + AirContent[4]);

int EnergyValues = 0;
output = null;
Double[] EnergyContent;
EnergyValues = environmentalImpactInfo.GetEnergyConsumption(out output);
EnergyContent = (Double[])output;
Debug.Print(" Energy consumption");
Debug.Print(" Material: " + EnergyContent[0]);
Debug.Print(" Use: " + EnergyContent[1]);
Debug.Print(" Transportation: " + EnergyContent[2]);
Debug.Print(" Manufacturing: " + EnergyContent[3]);
Debug.Print(" End of Life: " + EnergyContent[4]);

```

```

}

```

```

public SldWorks swApp;

```

```

}

```

```

}

```

Calculate Environmental Impact of Part (VBA)

This example shows how to calculate the environmental impact of a part.

```

'-----
' Preconditions:
' 1. Open:
'   public_documents\samples\tutorial\api\ZMZ7005 End Plate.sldprt
' 2. Open an Immediate window.
' 3. Run to the breakpoint and wait for the environmental impact results to
'   finish updating in the Sustainability Task Pane.
' 4. Press F5 to finish.
'
' Postconditions: Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----
Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Option Explicit

Sub main()

    Set swApp = Application.SldWorks

    Dim PDoc As SldWorks.ModelDoc2
    Set PDoc = swApp.ActiveDoc

    Dim sustain As sustainabilityLib.swgsustainabilityApp
    Set sustain = PDoc.Extension.GetSustainability

    ' Material
    Debug.Print "Material:"
    Dim materialInfo As sustainabilityLib.SustainabilityMaterial
    Set materialInfo = sustain.GetSustainabilityMaterial

    Dim MaterialClass As String
    MaterialClass = "Plastics"
    materialInfo.MaterialClass = MaterialClass
    Debug.Print " Material class: " & MaterialClass

    Dim MaterialName As String
    MaterialName = "ABS"
    materialInfo.MaterialName = MaterialName
    Debug.Print " Material name: " & MaterialName

    Debug.Print " % recycled: " & materialInfo.RecycledContent
    Debug.Print " Weight: " & materialInfo.Weight

    ' Manufacturing
    Debug.Print "Manufacturing:"
    Dim manufacturingInfo As sustainabilityLib.SustainabilityManufacturing
    Set manufacturingInfo = sustain.GetSustainabilityManufacturing

    Debug.Print " Region: " & manufacturingInfo.ManufacturingRegion
    Debug.Print " Built to last: " & manufacturingInfo.BuiltToLast
    Debug.Print " Process: " & manufacturingInfo.ManufacturingProcess
    Debug.Print " Primary electricity: " & manufacturingInfo.PrimaryElectricity
    Debug.Print " Primary natural gas: " & manufacturingInfo.PrimaryNaturalGas
    Debug.Print " Primary scrap rate: " & manufacturingInfo.PrimaryScrapRate
    Debug.Print " Paint: " & manufacturingInfo.Paint
    Debug.Print " Surface area: " & manufacturingInfo.SurfaceArea

    ' Transportation
    Debug.Print "Transportation:"
    Dim transInfo As sustainabilityLib.SustainabilityTransportation
    Set transInfo = sustain.GetSustainabilityTransportation

    Debug.Print " By train: " & transInfo.Train
    Debug.Print " By truck: " & transInfo.Truck
    Debug.Print " By boat: " & transInfo.Boat
    Debug.Print " By plane: " & transInfo.Plane

    ' Disposal at end of life
    Debug.Print "Disposal at end of life:"
    Dim eofInfo As sustainabilityLib.SustainabilityEndOfLife
    Set eofInfo = sustain.GetSustainabilityEndOfLife

    Debug.Print " % recycled: " & eofInfo.Recycled
    Debug.Print " % incinerated: " & eofInfo.Incinerated

```

```
Debug.Print " % landfill: " & eofInfo.Landfill
```

```
' Use
```

```
Debug.Print "Use:"
```

```
Dim partInfo As sustainabilityLib.SustainabilityPartUse
```

```
Set partInfo = sustain.GetSustainabilityPartUse
```

```
Debug.Print " Region of use: " & partInfo.Region
```

```
Stop
```

```
' Environmental impact
```

```
Debug.Print "Environmental impact:"
```

```
Dim environmentalImpactInfo As sustainabilityLib.SustainabilityEnvironmentalImpact
```

```
Set environmentalImpactInfo = sustain.GetSustainabilityEnvironmentalImpact
```

```
Dim CarbonValues As Integer
```

```
Dim CarbonContent As Variant
```

```
CarbonValues = environmentalImpactInfo.GetCurrentCarbonFootPrint(CarbonContent)
```

```
Debug.Print " Carbon footprint"
```

```
Debug.Print " Material: " & CarbonContent(0)
```

```
Debug.Print " Use: " & CarbonContent(1)
```

```
Debug.Print " Transportation: " & CarbonContent(2)
```

```
Debug.Print " Manufacturing: " & CarbonContent(3)
```

```
Debug.Print " End of Life: " & CarbonContent(4)
```

```
Dim WaterValues As Integer
```

```
Dim WaterContent As Variant
```

```
WaterValues = environmentalImpactInfo.GetWaterEutrophication(WaterContent)
```

```
Debug.Print " Water eutrophication"
```

```
Debug.Print " Material: " & WaterContent(0)
```

```
Debug.Print " Use: " & WaterContent(1)
```

```
Debug.Print " Transportation: " & WaterContent(2)
```

```
Debug.Print " Manufacturing: " & WaterContent(3)
```

```
Debug.Print " End of Life: " & WaterContent(4)
```

```
Dim AirValues As Integer
```

```
Dim AirContent As Variant
```

```
AirValues = environmentalImpactInfo.GetAirAcidification(AirContent)
```

```
Debug.Print " Air acidification"
```

```
Debug.Print " Material: " & AirContent(0)
```

```
Debug.Print " Use: " & AirContent(1)
```

```
Debug.Print " Transportation: " & AirContent(2)
```

```
Debug.Print " Manufacturing: " & AirContent(3)
```

```
Debug.Print " End of Life: " & AirContent(4)
```

```
Dim EnergyValues As Integer
```

```
Dim EnergyContent As Variant
```

```
EnergyValues = environmentalImpactInfo.GetEnergyConsumption(EnergyContent)
```

```
Debug.Print " Energy consumption"
```

```
Debug.Print " Material: " & EnergyContent(0)
```

```
Debug.Print " Use: " & EnergyContent(1)
```

```
Debug.Print " Transportation: " & EnergyContent(2)
```

```
Debug.Print " Manufacturing: " & EnergyContent(3)
```

```
Debug.Print " End of Life: " & EnergyContent(4)
```

```
End Sub
```

Calculate Environmental Impact of Part (VB.NET)

This example shows how to calculate the environmental impact of a part.

```

'-----
' Preconditions:
' 1. Add the SOLIDWORKS Sustainability primary interop assembly as
'    a reference (in the IDE's Project Explorer, right-click
'    the project name, select Add Reference, click the Browse tab,
'    navigate to the install_dir\api\redist folder and
'    select SolidWorks.Interop.sustainability.dll).
' 2. Open:
'    public_documents\samples\tutorial\api\ZM7005 End Plate.sldprt
' 3. Open an Immediate window.
' 4. Run to the breakpoint and wait for the environmental impact results to
'    finish updating in the Sustainability Task Pane.
' 5. Press F5 to finish.
'
' Postconditions: Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.sustainability
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModelDoc As ModelDoc2

    Sub main()

        Dim PDoc As ModelDoc2
        PDoc = swApp.ActiveDoc

        Dim sustain As swgsustainabilityApp
        sustain = PDoc.Extension.GetSustainability

        ' Material
        Debug.Print("Material:")
        Dim materialInfo As SustainabilityMaterial
        materialInfo = sustain.GetSustainabilityMaterial

        Dim MaterialClass As String
        MaterialClass = "Plastics"
        materialInfo.MaterialClass = MaterialClass
        Debug.Print(" Material class: " & MaterialClass)

        Dim MaterialName As String
        MaterialName = "ABS"
        materialInfo.MaterialName = MaterialName
        Debug.Print(" Material name: " & MaterialName)

        Debug.Print(" % recycled: " & materialInfo.RecycledContent)
        Debug.Print(" Weight: " & materialInfo.Weight)

        ' Manufacturing
        Debug.Print("Manufacturing:")
        Dim manufacturingInfo As SustainabilityManufacturing
        manufacturingInfo = sustain.GetSustainabilityManufacturing

        Debug.Print(" Region: " & manufacturingInfo.ManufacturingRegion)
        Debug.Print(" Built to last: " & manufacturingInfo.BuiltToLast)
        Debug.Print(" Process: " & manufacturingInfo.ManufacturingProcess)
        Debug.Print(" Primary electricity: " & manufacturingInfo.PrimaryElectricity)
        Debug.Print(" Primary natural gas: " & manufacturingInfo.PrimaryNaturalGas)
        Debug.Print(" Primary scrap rate: " & manufacturingInfo.PrimaryScrapRate)
        Debug.Print(" Paint: " & manufacturingInfo.Paint)
        Debug.Print(" Surface area: " & manufacturingInfo.SurfaceArea)

        ' Transportation
        Debug.Print("Transportation:")
        Dim transInfo As SustainabilityTransportation
        transInfo = sustain.GetSustainabilityTransportation

        Debug.Print(" By train: " & transInfo.Train)
        Debug.Print(" By truck: " & transInfo.Truck)
        Debug.Print(" By boat: " & transInfo.Boat)
        Debug.Print(" By plane: " & transInfo.Plane)

        ' Disposal at end of life
        Debug.Print("Disposal at end of life:")
        Dim eofInfo As SustainabilityEndOfLife
        eofInfo = sustain.GetSustainabilityEndOfLife
    
```

```

Debug.Print(" % recycled: " & eofInfo.Recycled)
Debug.Print(" % incinerated: " & eofInfo.Incinerated)
Debug.Print(" % landfill: " & eofInfo.Landfill)

' Use
Debug.Print("Use:")
Dim partInfo As SustainabilityPartUse
partInfo = sustain.GetSustainabilityPartUse

Debug.Print(" Region of use: " & partInfo.Region)

Stop

' Environmental impact
Debug.Print("Environmental impact:")
Dim environmentalImpactInfo As SustainabilityEnvironmentalImpact
environmentalImpactInfo = sustain.GetSustainabilityEnvironmentalImpact

Dim CarbonValues As Integer
Dim CarbonContent As Object = Nothing
CarbonValues = environmentalImpactInfo.GetCurrentCarbonFootPrint(CarbonContent)
Debug.Print(" Carbon footprint")
Debug.Print(" Material: " & CarbonContent(0))
Debug.Print(" Use: " & CarbonContent(1))
Debug.Print(" Transportation: " & CarbonContent(2))
Debug.Print(" Manufacturing: " & CarbonContent(3))
Debug.Print(" End of Life: " & CarbonContent(4))

Dim WaterValues As Integer
Dim WaterContent As Object = Nothing
WaterValues = environmentalImpactInfo.GetWaterEutrophication(WaterContent)
Debug.Print(" Water eutrophication")
Debug.Print(" Material: " & WaterContent(0))
Debug.Print(" Use: " & WaterContent(1))
Debug.Print(" Transportation: " & WaterContent(2))
Debug.Print(" Manufacturing: " & WaterContent(3))
Debug.Print(" End of Life: " & WaterContent(4))

Dim AirValues As Integer
Dim AirContent As Object = Nothing
AirValues = environmentalImpactInfo.GetAirAcidification(AirContent)
Debug.Print(" Air acidification")
Debug.Print(" Material: " & AirContent(0))
Debug.Print(" Use: " & AirContent(1))
Debug.Print(" Transportation: " & AirContent(2))
Debug.Print(" Manufacturing: " & AirContent(3))
Debug.Print(" End of Life: " & AirContent(4))

Dim EnergyValues As Integer
Dim EnergyContent As Object = Nothing
EnergyValues = environmentalImpactInfo.GetEnergyConsumption(EnergyContent)
Debug.Print(" Energy consumption")
Debug.Print(" Material: " & EnergyContent(0))
Debug.Print(" Use: " & EnergyContent(1))
Debug.Print(" Transportation: " & EnergyContent(2))
Debug.Print(" Manufacturing: " & EnergyContent(3))
Debug.Print(" End of Life: " & EnergyContent(4))

End Sub

Public swApp As SldWorks

End Class

```

Calculate Transformations in Part Example (C#)

This example shows how to calculate transformations between solid bodies in a part, which would make one solid body coincide with another solid body if the transformation matrix is applied.

```
//-----
// Preconditions:
// 1. Ensure that the specified part file to
//    open exists.
// 2. Open the Immediate window.
// 3. Run the macro.
//
// Postconditions:
// 1. Opens the specified part file.
// 2. Gets the solid bodies in the specified part file.
// 3. Iterates through the solid bodies in the part
//    and calculates transformations between solid
//    bodies.
// 4. Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CoincidenceCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            string file = null;
            int status = 0;
            int warnings = 0;

            file = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\routing-pipes\\routes\\Framework (Done).SLDPRT";
            swModel = (ModelDoc2)swApp.OpenDoc6(file, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref status, ref warnings);
            if (swModel == null)
                return;

            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            PartDoc swPart = default(PartDoc);
            swPart = (PartDoc)swModel;

            object[] bodies = null;
            Body2 swBody_A = default(Body2);
            Body2 swBody_B = default(Body2);

            //Get solid bodies
            swBodyType_e bodyType = default(swBodyType_e);
            bool visibleOnly = false;
            bodyType = swBodyType_e.swSolidBody;
            visibleOnly = true;
            bodies = (object[])swPart.GetBodies2((int)bodyType, visibleOnly);
            //Iterate through the solid bodies in the part
            //and calculate the transformation matrix between
            //swBody_A and swBody_B
            if (!((bodies == null)))
            {
                int i = 0;
                int j = 0;
                int maxIndex = 0;
                maxIndex = bodies.Length;
                for (i = 0; i <= maxIndex - 1; i++)
                {
                    swBody_A = (Body2)bodies[i];
                    for (j = 0; j <= maxIndex - 1; j++)
                    {
                        if (i != j)
                        {
                            swBody_B = (Body2)bodies[j];

                            MathTransform swXfm = default(MathTransform);
                            bool bResult = false;
                            swXfm = null;
                            bResult = false;
                            bResult = swBody_A.GetCoincidenceTransform2(swBody_B, out swXfm);

                            Debug.Print(i + " -> " + j, swBody_A.Name + " -> " + swBody_B.Name);

                            if (bResult != false)
                            {
                                Debug.Print("\n" + " Can coincide.");
                                double[] vXfm = null;
                                vXfm = (double[])swXfm.ArrayData;
                                Debug.Print("\n" + " Rotation:");
                                Debug.Print("\n" + " " + vXfm[0], vXfm[1], vXfm[2]);
                                Debug.Print("\n" + " " + vXfm[3], vXfm[4], vXfm[5]);
                                Debug.Print("\n" + " " + vXfm[6], vXfm[7], vXfm[8]);
                                Debug.Print("\n" + " Translation:");
                                Debug.Print("\n" + " " + vXfm[9], vXfm[10], vXfm[11]);
                                Debug.Print("\n" + " Scaling: " + vXfm[12]);
                            }
                            else
                            {
                                Debug.Print(" Cannot coincide.");
                            }
                        }
                    }
                }
            }
        }
    }
}
```

```
}  
  
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;  
}
```

Calculate Transformations in Part Example (VBA)

This example shows how to calculate transformations between solid bodies in a part, which would make one solid body coincide with another solid body if the transformation matrix is applied.

```

'-----
' Preconditions:
' 1. Ensure that the specified part file to
'    open exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Opens the specified part file.
' 2. Gets the solid bodies in the specified part file.
' 3. Iterates through the solid bodies in the part
'    and calculates transformations between solid
'    bodies.
' 4. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim file As String
    Dim status As Long
    Dim warnings As Long

    Set swApp = Application.SldWorks

    file = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\routing-pipes\routes\Framework (Done).SLDPRT"
    Set swModel = swApp.OpenDoc6(file, swDocPART, swOpenDocOptions_Silent, "", status, warnings)
    If swModel Is Nothing Then Exit Sub

    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    Dim swPart As SldWorks.PartDoc
    Set swPart = swModel

    Dim bodies As Variant
    Dim swBody_A As SldWorks.Body2
    Dim swBody_B As SldWorks.Body2

    'Get solid bodies
    Dim bodyType As swBodyType_e
    Dim visibleOnly As Boolean
    bodyType = swSolidBody
    visibleOnly = True
    bodies = swPart.GetBodies2(bodyType, visibleOnly)
    'Iterate through the solid bodies in the part
    'and calculate the transformation matrix between
    'swBody_A and swBody_B
    'coincide when possible
    If Not IsEmpty(bodies) Then
        Dim i As Long
        Dim j As Long
        Dim maxIndex As Long
        maxIndex = UBound(bodies)
        For i = 0 To maxIndex
            Set swBody_A = bodies(i)
            For j = 0 To maxIndex
                If i <> j Then
                    Set swBody_B = bodies(j)

                    Dim swXfm As MathTransform
                    Dim bResult As Boolean
                    Set swXfm = Nothing
                    bResult = False
                    bResult = swBody_A.GetCoincidenceTransform2(swBody_B, swXfm)

                    Debug.Print i & " -> " & j, swBody_A.Name & " -> " & swBody_B.Name

                    If bResult <> False Then
                        Debug.Print vbTab & "Can coincide."
                        Dim vXfm As Variant
                        vXfm = swXfm.ArrayData
                        Debug.Print vbTab & "Rotation:"
                        Debug.Print vbTab & vXfm(0), vXfm(1), vXfm(2)
                        Debug.Print vbTab & vXfm(3), vXfm(4), vXfm(5)
                        Debug.Print vbTab & vXfm(6), vXfm(7), vXfm(8)
                        Debug.Print vbTab & "Translation:"
                        Debug.Print vbTab & vXfm(9), vXfm(10), vXfm(11)
                        Debug.Print vbTab & "Scaling: " & vXfm(12)

                    Else
                        Debug.Print vbTab & "Cannot coincide."
                    End If
                End If
            Next j
        Next i
    End If
End Sub

```


Calculate Transformations in Part Example (VB.NET)

This example shows how to calculate transformations between solid bodies in a part, which would make one solid body coincide with another solid body if the transformation matrix is applied.

```
'-----
' Preconditions:
' 1. Ensure that the specified part file to
'    open exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Opens the specified part file.
' 2. Gets the solid bodies in the specified part file.
' 3. Iterates through the solid bodies in the part
'    and calculates transformations between solid
'    bodies.
' 4. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim file As String
        Dim status As Integer
        Dim warnings As Integer

        file = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\routing-pipes\routes\Framework (Done).SLDPRT"
        swModel = swApp.OpenDoc6(file, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", status, warnings)
        If swModel Is Nothing Then Exit Sub

        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        Dim swPart As PartDoc
        swPart = swModel

        Dim bodies As Object
        Dim swBody_A As Body2
        Dim swBody_B As Body2

        'Get solid bodies
        Dim bodyType As swBodyType_e
        Dim visibleOnly As Boolean
        bodyType = swBodyType_e.swSolidBody
        visibleOnly = True
        bodies = swPart.GetBodies2(bodyType, visibleOnly)
        'Iterate through the solid bodies in the part
        'and calculate the transformation matrix between
        'swBody_A and swBody_B
        If Not (IsNothing(bodies)) Then
            Dim i As Integer
            Dim j As Integer
            Dim j As Integer
            Dim maxIndex As Integer
            maxIndex = UBound(bodies)
            For i = 0 To maxIndex
                swBody_A = bodies(i)
                For j = 0 To maxIndex
                    If i <> j Then
                        swBody_B = bodies(j)

                        Dim swXfm As MathTransform
                        Dim bResult As Boolean
                        swXfm = Nothing
                        bResult = False
                        bResult = swBody_A.GetCoincidenceTransform2(swBody_B, swXfm)

                        Debug.Print(i & " -> " & j, swBody_A.Name & " -> " & swBody_B.Name)

                        If bResult <> False Then
                            Debug.Print(vbTab & "Can coincide.")
                            Dim vXfm As Object
                            vXfm = swXfm.ArrayData
                            Debug.Print(vbTab & "Rotation:")
                            Debug.Print(vbTab & vXfm(0), vXfm(1), vXfm(2))
                            Debug.Print(vbTab & vXfm(3), vXfm(4), vXfm(5))
                            Debug.Print(vbTab & vXfm(6), vXfm(7), vXfm(8))
                            Debug.Print(vbTab & "Translation:")
                            Debug.Print(vbTab & vXfm(9), vXfm(10), vXfm(11))
                            Debug.Print(vbTab & "Scaling: " & vXfm(12))

                        Else
                            Debug.Print(vbTab & "Cannot coincide.")
                        End If
                    End If
                Next j
            Next i
        End If

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Call Compiled HTML Help File Example (C#)

This example shows how to call a compiled HTML Help file.

```
'-----  
'  
' Preconditions: Specified compiled HTML Help file exists.  
'  
' Postconditions: Compiled HTML Help file is displayed.  
'  
'-----  
public bool OnHelp()  
{  
    string url = "E:/am/2005/C#/DisplayHelpFromPropMgr/apihelp.chm";  
  
    //Create a basic form to be the basis for the Help file  
    System.Windows.Forms.Form helpForm = new System.Windows.Forms.Form();  
    System.Windows.Forms.Help.ShowHelp(helpForm, url);  
    return true;  
}
```

Capture 3D View Example (C#)

This example shows how to capture the 3D view of a part or assembly and get its properties.

```
//-----
// Preconditions:
// 1. Open a part or assembly.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Captures 3DViewn.
// 2. Prints out all of the 3D View names.
// 3. Renames 3DViewn to Aleph.
// 4. Refreshes the model's 3D Views.
// 5. Observe Aleph on the 3DViews tab.
// 6. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace Capture3DView_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swDoc;
        ModelDocExtension swModelExt;
        View3D sw3DView;
        String[] viewNames;
        int v_counter;
        string modelBreakViewName = null;

        public void Main()
        {
            swDoc = (ModelDoc2)swApp.ActiveDoc;
            swModelExt = swDoc.Extension;

            sw3DView = swModelExt.Capture3DView();
            viewNames = (string[])swModelExt.Get3DViewNames();
            Debug.Print("3D Views:");
            for (v_counter = 0; v_counter <= viewNames.GetUpperBound(0); v_counter++) {
                Debug.Print(" " + viewNames[v_counter]);
            }

            sw3DView.Activate(true);
            sw3DView.Name = "Aleph";
            swModelExt.Refresh3DViews();
            Debug.Print("Number of 3D Views: " + swModelExt.Get3DViewCount());
            Debug.Print("Name of active 3D View: " + sw3DView.Name);
            Debug.Print("Configuration: " + sw3DView.ConfigurationName);
            Debug.Print("Display state: " + sw3DView.DisplayState);
            Debug.Print("Display mode as defined in swDisplayMode_e: " + sw3DView.DisplayMode);
            Debug.Print("Scale: " + sw3DView.Scale);
            modelBreakViewName = sw3DView.ModelBreakViewName;
            if (modelBreakViewName == null)
            {
                Debug.Print("No model break view");
            }
            else
            {
                Debug.Print("Model break view name: " + modelBreakViewName);
            }
        }

        public SldWorks swApp;
    }
}
```

Capture 3D View Example (VBA)

This example shows how to capture the 3D view of a part or assembly and get its properties.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Captures 3DViewn.
' 2. Prints out all of the 3D View names.
' 3. Renames 3DViewn to Aleph.
' 4. Refreshes the model's 3D Views.
' 5. Observe Aleph on the 3DViews tab.
' 6. Inspect the Immediate window.
'-----
```

```
Dim swApp As SldWorks.SldWorks
Dim swDoc As ModelDoc2
Dim swModelExt As ModelDocExtension
Dim sw3DView As View3D
Dim viewNames As Variant
Dim v_counter As Long
Dim modelBreakViewName As String
```

```
Option Explicit
Sub main()
```

```
    Set swApp = Application.SldWorks
    Set swDoc = swApp.ActiveDoc
    Set swModelExt = swDoc.Extension
```

```
    Set sw3DView = swModelExt.Capture3DView
    viewNames = swModelExt.Get3DViewNames
    Debug.Print "3D Views:"
    For v_counter = 0 To UBound(viewNames)
        Debug.Print "  " & viewNames(v_counter)
    Next
```

```
    sw3DView.Activate (True)
    sw3DView.Name = "Aleph"
    swModelExt.Refresh3DViews
    Debug.Print "Number of 3D Views: " & swModelExt.Get3DViewCount
    Debug.Print "Name of active 3D View: " & sw3DView.Name
    Debug.Print "Configuration: " & sw3DView.ConfigurationName
    Debug.Print "Display state: " & sw3DView.DisplayState
    Debug.Print "Display mode as defined in swDisplayMode_e: " & sw3DView.DisplayMode
    Debug.Print "Scale: " & sw3DView.Scale
    modelBreakViewName = sw3DView.modelBreakViewName
    If modelBreakViewName = "" Then
        Debug.Print "No model break view"
    Else
        Debug.Print "Model break view name: " & modelBreakViewName
    End If
```

```
End Sub
```

Capture 3D View Example (VB.NET)

This example shows how to capture the 3D view of a part or assembly and get its properties.

```

'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Captures 3DViewn.
' 2. Prints out all of the 3D View names.
' 3. Renames 3DViewn to Aleph.
' 4. Refreshes the model's 3D Views.
' 5. Observe Aleph on the 3DViews tab.
' 6. Inspect the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swDoc As ModelDoc2
    Dim swModelExt As ModelDocExtension
    Dim sw3DView As View3D
    Dim viewNames As Object
    Dim v_counter As Integer
    Dim modelBreakViewName As String = ""

    Sub main()

        swDoc = swApp.ActiveDoc
        swModelExt = swDoc.Extension

        sw3DView = swModelExt.Capture3DView
        viewNames = swModelExt.Get3DViewNames
        Debug.Print("3D Views:")
        For v_counter = 0 To UBound(viewNames)
            Debug.Print(" " & viewNames(v_counter))
        Next

        sw3DView.Activate(True)
        sw3DView.Name = "Aleph"
        swModelExt.Refresh3DViews()
        Debug.Print("Number of 3D Views: " & swModelExt.Get3DViewCount)
        Debug.Print("Name of active 3D View: " & sw3DView.Name)
        Debug.Print("Configuration: " & sw3DView.ConfigurationName)
        Debug.Print("Display state: " & sw3DView.DisplayState)
        Debug.Print("Display mode as defined in swDisplayMode_e: " & sw3DView.DisplayMode)
        Debug.Print("Scale: " & sw3DView.Scale)
        modelBreakViewName = sw3DView.ModelBreakViewName
        If modelBreakViewName = "" Then
            Debug.Print("No model break view")
        Else
            Debug.Print("Model break view name: " & modelBreakViewName)
        End If

    End Sub

    Public swApp As SldWorks

End Class

```

Change Active Tab in Manager Pane Example (C#)

This example shows how to change the active tab in the Manager Pane.

```
//-----
// Preconditions:
// 1. Open an assembly and click the FeatureManager design tree tab
//    in the Manager Pane.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Ensures that the Manager Pane is visible.
// 2. Traverses the tabs in the Manager Pane:
//    a. Changes the active tab.
//    b. Fires a notification.
// 3. Examine the Immediate window and Manager Pane.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Collections;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public AssemblyDoc swAssy;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExtension = default(ModelDocExtension);
            ModelViewManager swModelViewManager = default(ModelViewManager);
            object[] tabs = null;
            int nbrTabs = 0;
            int nbrTab = 0;
            Hashtable openAssy = default(Hashtable);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swModelDocExtension = (ModelDocExtension)swModel.Extension;

            //Set up events
            openAssy = new Hashtable();
            AttachEventHandlers();

            swModelViewManager = (ModelViewManager)swModel.ModelViewManager;

            //Ensure that Manager Pane is visible
            swModelDocExtension.HideFeatureManager(false);

            //Get the tabs in the Manager Pane
            tabs = (object[])swModelViewManager.GetFeatureManagerTabs();
            nbrTabs = tabs.GetLength(0);
            Debug.Print("Number of tabs in Manager Pane: " + nbrTabs);

            //Traverse the tabs in the Manager Pane and change the active tab
            for (nbrTab = 0; nbrTab < nbrTabs; nbrTab++)
            {
                swModelViewManager.ActiveFeatureManagerTabIndex = nbrTab;
            }

            Debug.Print("");

            Debug.Print("FeatureManager design tree tab index: " + swModelViewManager.GetFeatureManagerTreeTabIndex());
            Debug.Print("PropertyManager tab index: " + swModelViewManager.GetPropertyManagerTabIndex());
            Debug.Print("ConfigurationManager tab index: " + swModelViewManager.GetConfigurationManagerTabIndex());
            Debug.Print("DimXpertManager tab index: " + swModelViewManager.GetDimXpertManagerTabIndex());
            Debug.Print("DisplayManager tab index: " + swModelViewManager.GetDisplayManagerTabIndex());
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAssy.FeatureManagerTabActivatedNotify += this.swAssy_FeatureManagerTabActivatedNotify;
        }

        public int swAssy_FeatureManagerTabActivatedNotify(int commandIndex, string commandTabName)
        {
            Debug.Print("Activated tab " + commandIndex + " whose name is " + commandTabName);
            return 0;
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Change Active Tab in Manager Pane Example (VBA)

This example shows how to change the active tab in the Manager Pane.

```

'-----
' Preconditions:
' 1. Open an assembly and click the FeatureManager design tree tab
'   in the Manager Pane.
' 2. In the SOLIDWORKS Microsoft Visual Basic for Applications IDE:
'   a. Copy Main to the main module.
'   b. Click Insert > Class Module and copy Class1 to that module.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Ensures that the Manager Pane is visible.
' 2. Traverses the tabs in the Manager Pane:
'   a. Changes the active tab.
'   b. Fires a notification.
' 3. Examine the Immediate window and Manager Pane.
'-----
'Main

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExtension As SldWorks.ModelDocExtension
Dim swAssembly As SldWorks.AssemblyDoc
Dim swModelViewManager As SldWorks.ModelViewManager
Dim tabs As Variant
Dim nbrTabs As Long
Dim nbrTab As Long
Dim swAssemblyEvents As Class1

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssembly = swModel
    Set swModelDocExtension = swModel.Extension

    'Set up events
    Set swAssemblyEvents = New Class1
    Set swAssemblyEvents.swAssembly = swApp.ActiveDoc

    Set swModelViewManager = swModel.ModelViewManager

    'Ensure that Manager Pane is visible
    swModelDocExtension.HideFeatureManager (False)

    'Get the tabs in the Manager Pane
    tabs = swModelViewManager.GetFeatureManagerTabs
    nbrTabs = UBound(tabs) + 1
    Debug.Print "Number of tabs in Manager Pane: " & nbrTabs

    'Traverse the tabs in the Manager Pane and change the active tab
    For nbrTab = 0 To UBound(tabs)
        swModelViewManager.ActiveFeatureManagerTabIndex = nbrTab
    Next nbrTab

    Debug.Print ""

    'Get each tab's index
    Debug.Print "FeatureManager design tree tab index: " & swModelViewManager.GetFeatureManagerTreeTabIndex
    Debug.Print "PropertyManager tab index: " & swModelViewManager.GetPropertyManagerTabIndex
    Debug.Print "ConfigurationManager tab index: " & swModelViewManager.GetConfigurationManagerTabIndex
    Debug.Print "DimXpertManager tab index: " & swModelViewManager.GetDimXpertManagerTabIndex
    Debug.Print "DisplayManager tab index: " & swModelViewManager.GetDisplayManagerTabIndex

End Sub

Back to top

'Class1
Option Explicit

Public WithEvents swAssembly As SldWorks.AssemblyDoc

Public Function swAssembly_FeatureManagerTabActivatedNotify(ByVal commandIndex As Long, ByVal commandTabName As String) As Long
    Debug.Print "Activated tab " & commandIndex & " whose name is " & commandTabName
End Function

Back to top

```

Change Active Tab in Manager Pane Example (VB.NET)

This example shows how to change the active tab in the Manager Pane.

```
'-----
' Preconditions:
' 1. Open an assembly and click the FeatureManager design tree tab
'    in the Manager Pane.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Ensures that the Manager Pane is visible.
' 2. Traverses the tabs in the Manager Pane:
'    a. Changes the active tab.
'    b. Fires a notification.
' 3. Examine the Immediate window and Manager Pane.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
Imports System.Collections

Partial Class SolidWorksMacro

    Public WithEvents swAssy As AssemblyDoc

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExtension As ModelDocExtension
        Dim swModelViewManager As ModelViewManager
        Dim tabs() As Object
        Dim nbrTabs As Integer
        Dim nbrTab As Integer
        Dim openAssy As Hashtable

        swModel = swApp.ActiveDoc
        swAssy = swModel
        swModelDocExtension = swModel.Extension

        'Set up events
        openAssy = New Hashtable
        AttachEventHandlers()

        swModelViewManager = swModel.ModelViewManager

        'Ensure that Manager Pane is visible
        swModelDocExtension.HideFeatureManager(False)

        'Get the tabs in the Manager Pane
        tabs = swModelViewManager.GetFeatureManagerTabs
        nbrTabs = UBound(tabs) + 1
        Debug.Print("Number of tabs in Manager Pane: " & nbrTabs)

        'Traverse the tabs in the Manager Pane and change the active tab
        For nbrTab = 0 To UBound(tabs)
            swModelViewManager.ActiveFeatureManagerTabIndex = nbrTab
        Next

        Debug.Print("")

        Debug.Print("FeatureManager design tree tab index: " & swModelViewManager.GetFeatureManagerTreeTabIndex)
        Debug.Print("PropertyManager tab index: " & swModelViewManager.GetPropertyManagerTabIndex)
        Debug.Print("ConfigurationManager tab index: " & swModelViewManager.GetConfigurationManagerTabIndex)
        Debug.Print("DimXpertManager tab index: " & swModelViewManager.GetDimXpertManagerTabIndex)
        Debug.Print("DisplayManager tab index: " & swModelViewManager.GetDisplayManagerTabIndex)

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swAssy.FeatureManagerTabActivatedNotify, AddressOf Me.swAssy_FeatureManagerTabActivatedNotify
    End Sub

    Public Function swAssy_FeatureManagerTabActivatedNotify(ByVal commandIndex As Integer, ByVal commandTabName As String) As Integer
        Debug.Print("Activated tab " & commandIndex & " whose name is " & commandTabName)
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```